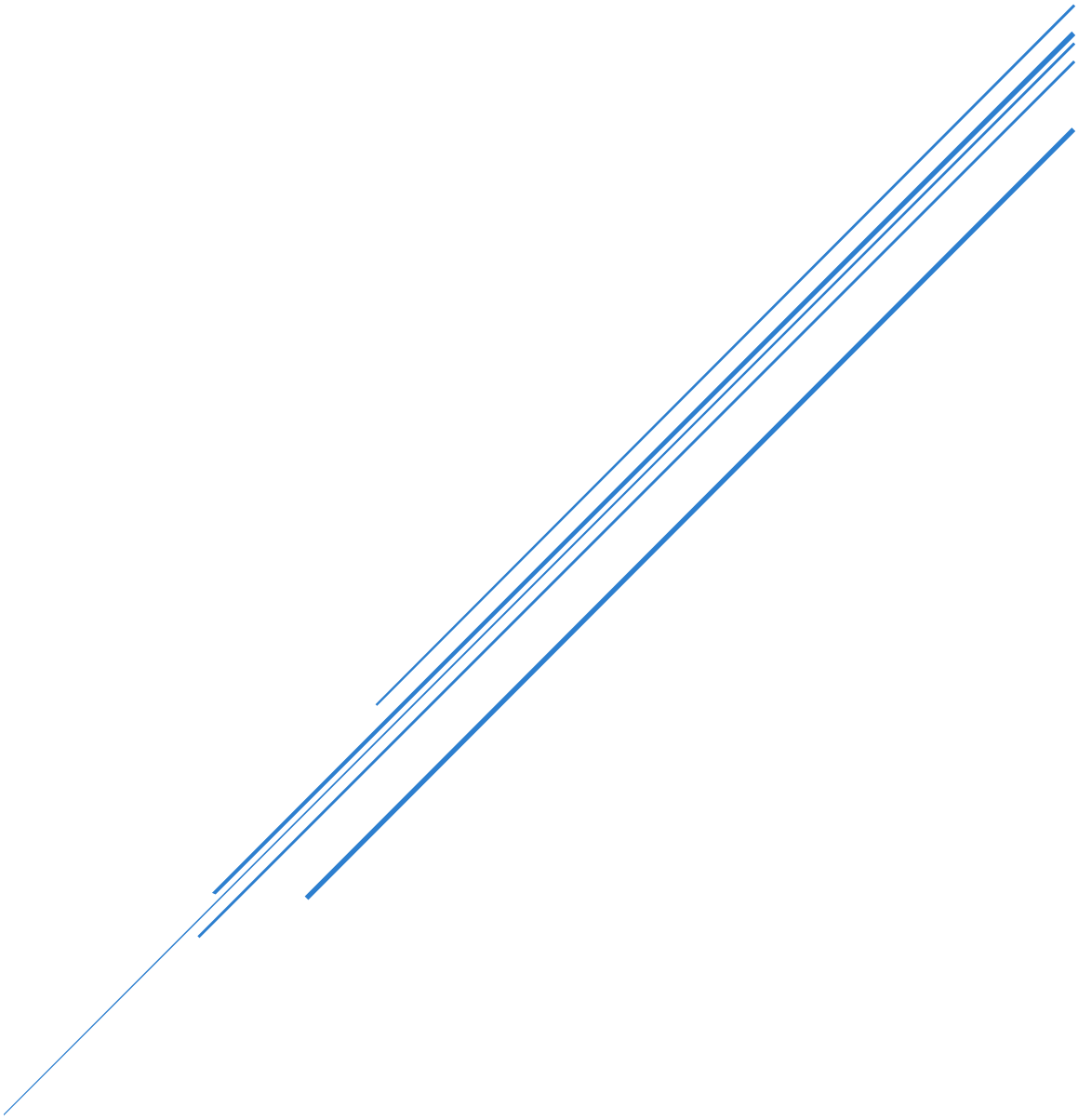
DYNAMIC PORTFOLIO ALLOCATION USING DEEP REINFORCEMENT LEARNING

ECE556



Taimoor Shahzad

Table of Contents

1. Abstract 1
2. Introduction 1
3. Problem Formulation 2
   3.1 Agent and Environment 2
   3.2 State Space 3
   3.3 Action Space 3
   3.4 Reward Function 3
   3.5 Transition Dynamics 4
4. Problem Classification 6
   4.1 Episodic vs. Continuous 6
   4.2 Deterministic vs. Stochastic Transitions 6
   4.3 Finite vs. Continuous State and Action Spaces 7
5. Solution Methods 7
   5.1 Inapplicability of Tabular Methods & Applicability of Policy-Based Methods 8
   5.2 Justification for Recurrent PPO 8
   5.3 Recurrent PPO Pseudocode 8
   5.4 Other Applicable RL Algorithms 10
   5.5 Role of Markowitz Model (MVO) 10
6. Implementation & Results 11
   6.1 Training Progression and Policy Dynamics 11
   6.2 Learned Policy Behavior 14
   6.3 Test Set Evaluation 14
   6.4 Comparison with Baseline (Markowitz) 16
7. Conclusion 17

References 19

Appendix 20
   A. Recurrent PPO Architecture Diagram 20
   B. Recurrent PPO Workflow Diagram 21
   C. Environment Setup & Key Hyperparameters 22
   D. Cumulative Return Plots for 10 Training Rounds 24
   E. Portfolio Weights Plots for 10 Training Rounds 29

1. Abstract

This project addresses the challenge of dynamic portfolio allocation in financial markets by formulating it as a reinforcement learning (RL) problem and evaluating two contrasting approaches: the traditional Markowitz Mean-Variance Optimization (MVO)[1] and a modern deep reinforcement learning method, Proximal Policy Optimization (PPO)[2] augmented with Long Short-Term Memory (LSTM)[3] networks. The goal is to optimize investment strategies over time by learning asset allocations that balance expected returns with risk and transaction costs.

The problem is modeled as a Markov Decision Process (MDP), where the environment consists of log-returns of five major stocks (AAPL, MSFT, TSLA, AMZN, and GOOG) over a ten-year period. The agent observes historical price data and allocates weights to each asset at every time step, with the objective of maximizing cumulative returns while incorporating realistic constraints such as transaction fees and allocation penalties.

A recurrent PPO agent capable of leveraging temporal dependencies was implemented, and its performance was compared against a conventional rolling-window MVO strategy. Experimental evaluation was conducted across training, validation, and test datasets, using metrics such as Sharpe Ratio and cumulative returns to assess performance. The results demonstrate that while MVO offers stable and interpretable baseline performance, the PPO-LSTM agent learns more adaptive strategies, outperforming MVO on unseen test data in terms of risk-adjusted returns. This study highlights the potential of reinforcement learning in financial decision-making and lays the foundation for future improvements involving market dynamics and risk modeling.

2. Introduction

In the ever-evolving landscape of financial markets, portfolio management remains a critical yet complex task. Investors and institutions are persistently seeking strategies that can adapt to market volatility, minimize risk, and optimize returns. Traditional approaches such as the Markowitz Mean-Variance Optimization (MVO) model have long provided a mathematically grounded framework for selecting asset allocations by balancing expected returns against risk through the covariance of returns[1]. While MVO has served as a foundational tool in modern portfolio theory, it is inherently static and relies on the assumption of stationarity and complete knowledge of return distributions. These limitations raise

concerns in dynamic, high-frequency trading environments where the market behavior is neither stable nor predictable.

In contrast, Reinforcement Learning (RL) offers a promising alternative by modeling portfolio allocation as a sequential decision-making process. By interacting with a simulated market environment, an RL agent can learn to adjust its strategy over time based on the observed outcomes of its actions. This adaptability makes RL particularly suited to financial contexts where future returns are uncertain and influenced by temporal dependencies. Among RL algorithms, Proximal Policy Optimization (PPO) has emerged as a robust method that balances exploration and exploitation while ensuring stable policy updates[2]. When augmented with recurrent architectures such as Long Short-Term Memory (LSTM) networks, PPO is capable of capturing patterns across time, which is crucial for financial time series prediction[3].

This project aims to implement and compare these two paradigms (MVO and Recurrent PPO) on the problem of dynamic portfolio allocation using historical stock data from prominent technology companies. By evaluating their performance across training, validation, and test datasets, the project seeks to explore the trade-offs between model simplicity, interpretability, and adaptive intelligence. Through this comparative study, I aim to contribute a practical perspective on how reinforcement learning can complement or surpass traditional methods in complex real-world financial applications.

3. Problem Formulation

The problem of dynamic portfolio allocation is modeled as a Markov Decision Process (MDP) to enable the use of reinforcement learning for learning optimal trading policies. The goal is to adjust asset allocations over time in a way that maximizes long-term returns while accounting for risk and transaction costs. This section defines the MDP components: the agent, environment, state space, action space, reward function, and transition dynamics.

3.1 Agent and Environment

The agent in this setting is a policy network trained using Recurrent Proximal Policy Optimization (Recurrent PPO). This policy takes as input a historical window of asset returns and outputs a portfolio allocation vector. The agent interacts with a custom environment that simulates financial market behavior using historical price returns of five major assets (AAPL, MSFT, TSLA, AMZN, and GOOG) over the period 2015–2024. The environment is responsible for returning observations, calculating rewards based on the agent's portfolio decisions, and advancing the market to the next time step. It ensures the interaction adheres to episodic constraints, such as a fixed trading horizon per episode.

3.2 State Space

The state at each time step is defined as a matrix of historical returns over a fixed window. In this implementation, a 25-day window of returns for all five assets is used. To feed the state into the neural network, the matrix is flattened into a continuous vector of size 125 (window size x no. of assets = 25 × 5). Formally, the state space is:

$$S \subseteq \mathbb{R}^{25\times5}$$

This representation allows the policy to infer temporal patterns and volatility trends in the recent market history.

3.3 Action Space

The action is a continuous allocation vector $w_t \in \mathbb{R}^5$, where each element represents the proportion of capital assigned to one of the five assets. The vector is normalized such that:

$$\sum_{i=1}^{5} w_t^{(i)} = 1, \quad w_t^{(i)} \geq 0$$

The sum of weights inside the asset allocation vector is equal to 1. This ensures the agent allocates 100% of the capital to the assets in the portfolio.

3.4 Reward Function

The reward at each step is designed to promote risk-adjusted returns while discouraging excessive rebalancing. It is computed as a weighted combination of daily portfolio return and the Sharpe ratio[4], with penalties applied for high portfolio turnover and allocation changes[5]. In this project, the Sharpe ratio was used as a key component of the reward function to introduce risk-awareness into the reinforcement learning policy. It quantifies the risk-adjusted return by measuring the average return earned in excess of the risk-free rate per unit of volatility. The theoretical formula for the Sharpe ratio is:

$$Sharpe = \frac{E[R_t] - R_f}{\sigma(R_t)}$$

Where:

- $E[R_t]\ is\ expected\ return$
- $R_f\ is\ the\ risk - free\ rate\ (assumed\ zero\ in\ this\ setup)$
- $\sigma(R_t)\ is\ the\ standard\ deviation\ of\ returns.$

Our environment generates daily returns, therefore the Sharpe ratio needs to be annualized to make it comparable with financial industry standards. This is achieved by multiplying the daily Sharpe ratio with the square root of the number of trading days in a year, typically 252:

$$Sharpe_{annualized} = \frac{E[R_t]}{\sigma(R_t)} \times \sqrt{252}$$

Let w_t be the allocation vector and r_{t+1} be the next-day asset returns. The net reward R_t is defined as:

$$R_t = \lambda_1.(w_t^T r_{t+1}) + \lambda_2.Sharpe_t - \lambda_3.Turnover_t - \lambda_4.Penalty_t$$

Where:

- $w_t^T r_{t+1}$ is daily portfolio return
- $Turnover_t = \sum|w_t - w_{t-1}|$
- $Penalty_t = \sum \| w_t - w_{t-1} \|^2$
- λ_1, λ_2 *are reward blend weights and* $\lambda_1 = 1 - \lambda_2$
- λ_3, λ_4 *are coefficients for transaction cost and allocation penalty*

This formulation introduces risk-awareness into the policy while encouraging stability in allocations[6].

- **Sharpe ratio term ($\lambda_2.Sharpe_t$)**: Promotes risk-adjusted returns, rather than blindly maximizing raw gains. A higher Sharpe ratio implies that the agent is able to achieve better returns per unit of risk, which is crucial in volatile financial markets.
- **Turnover penalty ($\lambda_3.Turnover_t$)**: Penalizes frequent large reallocations of portfolio weights. High turnover can lead to excessive transaction costs in real-world settings and may signal overfitting to short-term fluctuations.
- **Allocation penalty ($\lambda_4.Penalty_t$)**: Discourages abrupt changes in asset weights. It ensures smooth transitions over time, which improves policy stability and avoids oscillatory behavior, which can degrade both learning and real-world performance.

3.5 Transition Dynamics

The transition dynamics define how the environment evolves from one state to another given the agent's action. At each time step t, the environment transitions to a new state based on the most recent historical returns and the action taken by the agent. Specifically, given the allocation vector w_t , the next state

incorporates the new market return r_{t+1}, which is drawn from a preloaded historical dataset and appended to the return window, forming the new state. The action does not influence the return itself, but it affects the reward through portfolio performance. The next state is constructed by shifting the return window forward by one day and appending the newly observed return r_{t+1} . This mirrors how rolling windows of financial time series evolve, capturing realistic market dynamics where the same action can lead to different future returns depending on market conditions.

Each episode is defined to last 180 trading days, consistent with the evaluation framework used in both the Recurrent PPO model and the baseline Markowitz model. This episodic setup enables the evaluation of learned policies over multiple non-overlapping time windows across the training, validation, and test datasets.

Example Environment Setup:

- **Time Step:** 1 day, **Window Size:** 3 trading days, **Number of Assets:** 5 (AST1, AST2, AST3, AST4, AST5), **Current Day:** t = 3 (window includes Day 1 to Day 3)
- Current State S_t

Day	AST1	AST2	AST3	AST4	AST5
D1	0.010	-0.015	0.005	0.002	0.012
D2	-0.008	0.011	-0.007	0.004	-0.002
D3	0.013	0.000	0.009	-0.001	0.006

Table 1 - State Space at time t (Transition Dynamics Example)

Flattened State Vector (S_t) =
[0.010,−0.015,0.005,0.002,0.012,−0.008,0.011,−0.007,0.004,−0.002,0.013,0.000,0.009,−0.001,0.006]

- Action: The agent allocates weights across the 5 assets (must sum to 1).

$$w_t = [0.25,0.20,0.15,0.10,0.30]$$

- Observed Return at Day 4: $r_{t+1} = [0.017,-0.006,0.011,-0.002,0.009]$
- Daily Portfolio Return Calculation: $w_t^T r_{t+1} = 0.0072$
- Reward Calculation (R_t): Let,

Previous allocation weights: $w_{t-1} = [0.20,0.20,0.20,0.20,0.20]$

Coefficients: $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ = 0.5, 0.5, 0.1, 0.2

$Sharpe_t = 1.4$

So, $Turnover_t = \sum|w_t - w_{t-1}| = 0.25$ & $Penalty_t = \sum \| w_t - w_{t-1} \|^2 = 0.025$

Hence,

$R_t = 0.5\times0.0072 + 0.5\times1.4 - 0.1\times0.25 - 0.2\times0.025 = 0.6736$

Next State S_{t+1} (Drop Day 1 returns and add Day 4 return)

Day	AST1	AST2	AST3	AST4	AST5
D2	-0.008	0.011	-0.007	0.004	-0.002
D3	0.013	0.000	0.009	-0.001	0.006
D4	0.017	-0.006	0.011	-0.002	0.009

Table 2 - State Space at time t+1 (Transition Dynamics Example)

4. Problem Classification

In this project, the portfolio optimization task was framed as a Markov Decision Process (MDP), where the agent learns to make sequential asset allocation decisions based on observed historical returns. The problem is categorized along multiple dimensions, each discussed below with appropriate justification.

4.1 Episodic vs. Continuous

The portfolio allocation task is treated as an episodic MDP, where each episode corresponds to a fixed-length investment horizon. Specifically, an episode spans 180 trading days, after which the environment is reset to a new initial window sampled from the historical market data. This structure allows for repeated training and evaluation across multiple rolling windows in the dataset and helps to mitigate overfitting to a single time series path. Episodic formulation is particularly suitable in financial contexts where performance is evaluated over distinct time horizons such as quarters or fiscal years.

4.2 Deterministic vs. Stochastic Transitions

The classification of the MDP's transition function requires careful distinction between implementation behavior and real-world interpretation. From an implementation standpoint, the transition function is deterministic. Given a fixed dataset of historical returns, the next state is deterministically computed by

shifting the input window forward by one day and appending the next observed return. This process does not involve sampling from any probabilistic distribution. As a result, for the same initial state and time index, the environment will always produce the same next state:

$$S_{t+1} = f(S_t) \text{ Deterministic}$$

However, from a modeling perspective, financial markets are inherently stochastic. The historical returns used in the simulation represent just one realization from a potentially infinite set of possible outcomes. If the model were deployed live or trained on sampled trajectories from a generative process, the transitions would exhibit true randomness. Thus, our MDP's transition function is best described as:

- Deterministic in implementation, to ensure reproducibility and fixed benchmarking.
- Stochastic in interpretation, as it models an exogenous time series reflecting real-world uncertainty.

This distinction aligns with recent reinforcement learning literature on financial environments where historical datasets are used for both training and evaluation (e.g., Jiang et al., 2017; Moody & Saffell, 2001) without probabilistic market simulation.

4.3 Finite vs. Continuous State and Action Spaces

- **State Space**: The state space is continuous and finite-dimensional. Each state consists of a sequence of past returns for multiple assets over a sliding window (e.g., a 3-day window with 5 assets yields a 15-dimensional continuous vector). Although bounded by the size of the historical dataset, the actual values are continuous, representing real-valued percentage returns.
- **Action Space**: The action space continuous and infinite. At each time step, the agent outputs a portfolio allocation vector $w_t \in \mathbb{R}^n$, where n is the number of assets. The vector is constrained such that $\sum w_t = 1$, and $w_t \geq 0$, representing long-only, fully invested portfolios. This continuous formulation allows the agent to explore a wide range of portfolio combinations and fine-tune allocations.

5. Solution Methods

This portfolio allocation task is modeled as a high-dimensional, continuous control problem under partial observability, where the agent must learn to allocate capital across multiple assets based on historical market information. Due to the nature of the state and action spaces, as well as the need to capture temporal dependencies, policy-based reinforcement learning methods are best suited to address this problem.

5.1 Inapplicability of Tabular Methods & Applicability of Policy-Based Methods

Classical algorithms such as SARSA, Q-learning, and Expected SARSA are designed for discrete and finite environments. These methods rely on tabular representations of state-action pairs, which are impractical for this task due to the following reasons:

- The state space is continuous and high-dimensional, consisting of recent return sequences across multiple assets.
- The action space comprises portfolio weight vectors that are real-valued and continuous, resulting in infinitely many possible actions.

To handle continuous spaces, policy gradient methods are preferred. These algorithms directly parameterize and optimize the policy. Among such methods, Proximal Policy Optimization (PPO) has gained wide adoption due to its balance between stability and performance[7]. PPO improves training stability by using a clipped objective that constrains policy updates within a trust region. This prevents large, destabilizing policy changes during training.

5.2 Justification for Recurrent PPO

While PPO is effective under full observability, financial markets exhibit long-range temporal dependencies and partial observability. The agent only observes a short window of past returns, which may not capture the full context of market dynamics. To address this, Recurrent PPO (R-PPO), which integrates a recurrent network (LSTM) into both the policy and value function approximators[8] was used. This allows the agent to:

- Maintain an internal memory state
- Learn temporal patterns and dependencies over time
- Handle partial observability more effectively

This structure is particularly useful for sequential decision-making in financial markets where history matters, but cannot be fully captured by fixed-length observation windows alone.

Refer to Appendix A. for system level architecture diagram and Appendix B. for code workflow diagram.

5.3 Recurrent PPO Pseudocode

1. Import and preprocess data

   a. Download daily closing prices of selected assets from yfinance

b. Calculate log returns and normalize them

c. Split data into train, validation, and test sets

2. Define custom Portfolio Environment

a. Set action space as continuous allocation vector (size = #assets)

b. Define state as flattened return history over a rolling window

c. In step():

i. Normalize action to sum to 1

ii. Compute portfolio return using dot product with asset returns

iii. Apply transaction cost and allocation change penalty

iv. Compute Sharpe ratio over recent window

v. Blend Sharpe and return into reward function

vi. Update weights and advance environment to next time step

3. Initialize training and validation environments with “DummyVecEnv”

4. Train the R-PPO agent

For each training seed:

a. Initialize R-PPO with LSTM-based policy (“MlpLstmPolicy”)

b. Train in multiple rounds with early stopping:

i. In each round:

- Train for N timesteps using “.learn()”

- Evaluate on validation environment:

▪ Compute portfolio returns and Sharpe ratio

▪ Log performance metrics and visualize

ii. Save best-performing model based on validation Sharpe

5. Select the best seed and corresponding model based on highest validation Sharpe

6. Evaluate final model on test environment

   a. Rollout policy with LSTM state tracking

   b. Record test returns and asset allocation weights

   c. Compute and plot cumulative return and weight trajectories

7. Report test Sharpe ratio and final performance plots

5.4 Other Applicable RL Algorithms

In addition to PPO and its recurrent variant, several other algorithms are applicable to this problem domain:

- **Deep Deterministic Policy Gradient (DDPG):** Suitable for continuous action spaces. However, it is more sensitive to hyperparameters and exploration noise[9], which can be problematic in financial settings.
- **Soft Actor-Critic (SAC):** Offers sample efficiency and stability by maximizing a stochastic policy's entropy[10]. It is well-suited for continuous control and has shown strong performance in financial applications.
- **Twin Delayed DDPG (TD3):** Addresses overestimation bias[11] in DDPG through double Q-learning and delayed updates. While effective, it can be more complex to tune and less interpretable than PPO variants.

Despite their merits, PPO and R-PPO were selected for this project due to their empirical reliability, lower sensitivity to tuning, and successful application in many continuous control problems.

5.5 Role of Markowitz Model (MVO)

Although not a reinforcement learning algorithm, the Markowitz Mean-Variance Optimization (MVO) model[12] is included as a baseline for comparative evaluation. MVO solves the allocation problem through convex optimization by maximizing the expected return minus a penalty for portfolio variance. The closed-form formulation serves as a reference point for:

- Evaluating the ability of RL agents to outperform classical financial strategies
- Comparing performance on risk-adjusted metrics such as the Sharpe ratio
- Benchmarking behavior under realistic transaction cost and allocation constraints

This baseline helps to contextualize the performance gains achieved by the learning-based approach.

6. Implementation & Results

This section presents the implementation strategy of the Recurrent Proximal Policy Optimization (R-PPO) algorithm for portfolio management, along with a detailed analysis of its learning behavior and empirical performance on the validation and test dataset. The R-PPO model was implemented using the Recurrent PPO algorithm from the SB3-Contrib module [12], built on top of Stable-Baselines3[13]. This setup enabled recurrent policy learning via LSTM networks in a financial market simulation where an agent dynamically rebalances a portfolio of five assets: Apple, Microsoft, Tesla, Amazon, and Google. The core components of the implementation are as follows:

- **Observation space:** A 125-dimensional continuous state constructed by flattening a 25-day rolling window of standardized log-returns across 5 assets.
- **Action space:** A continuous 5-dimensional vector representing portfolio weights. The actions are normalized to sum to 1.
- **Reward function:** A weighted combination of Sharpe ratio and immediate log-return, with penalties for excessive turnover and allocation shifts.
- **Environment design:** A custom PortfolioEnv built using the OpenAI Gym interface[14], simulating 180-day episodes.
- **Training horizon:** The training data ranged from 2015 to 2020, and the model was trained for multiple rounds, each consisting of 50,000 timesteps. Early stopping was employed if validation Sharpe ratio plateaued for 4 consecutive rounds.
- **Validation & testing:** The validation data ranged from 2021 to 2023 while the year 2024 was held out for test evaluation. Model checkpoints were evaluated on a validation set, and the best-performing policy (by Sharpe ratio) was selected for out-of-sample testing.

Refer to Appendix C. for detailed Environment Setup & Key Hyperparameters.

6.1 Training Progression and Policy Dynamics

Training was performed using a fixed seed (25). In early rounds, cumulative returns were unstable (Figure 1), but beginning from Round 6, the policy stabilized and performance consistently improved (Figure 2)

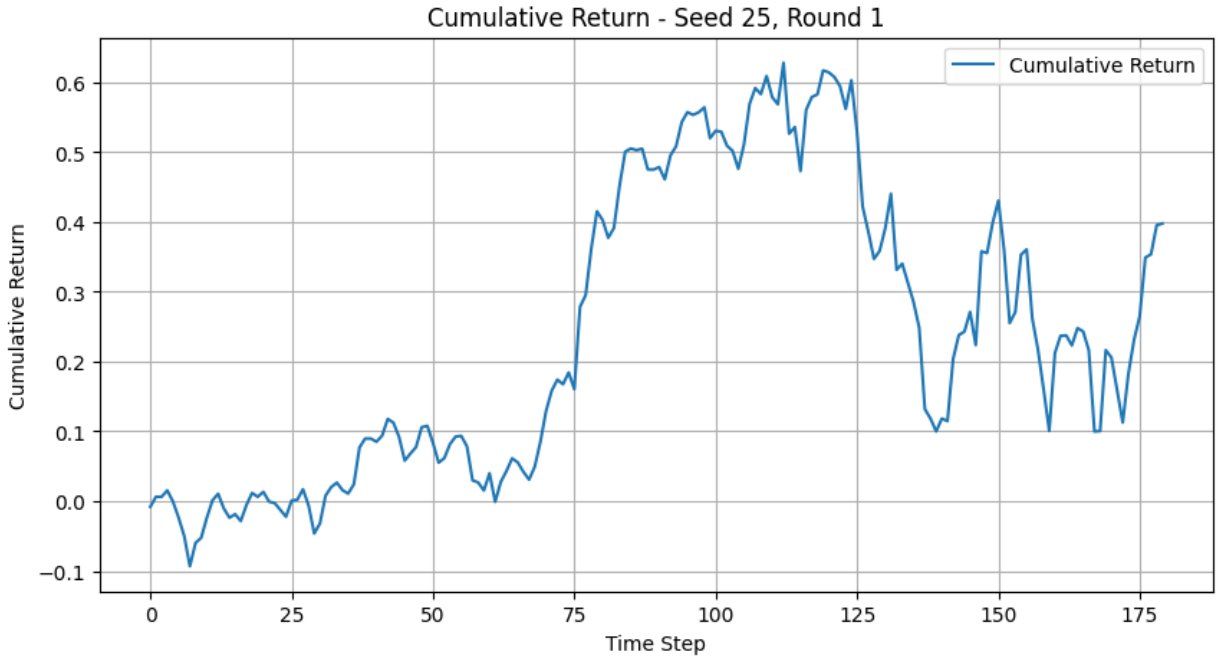


Figure 1 - Cumulative Portfolio Return Round 1 (Unstable) - Training Set

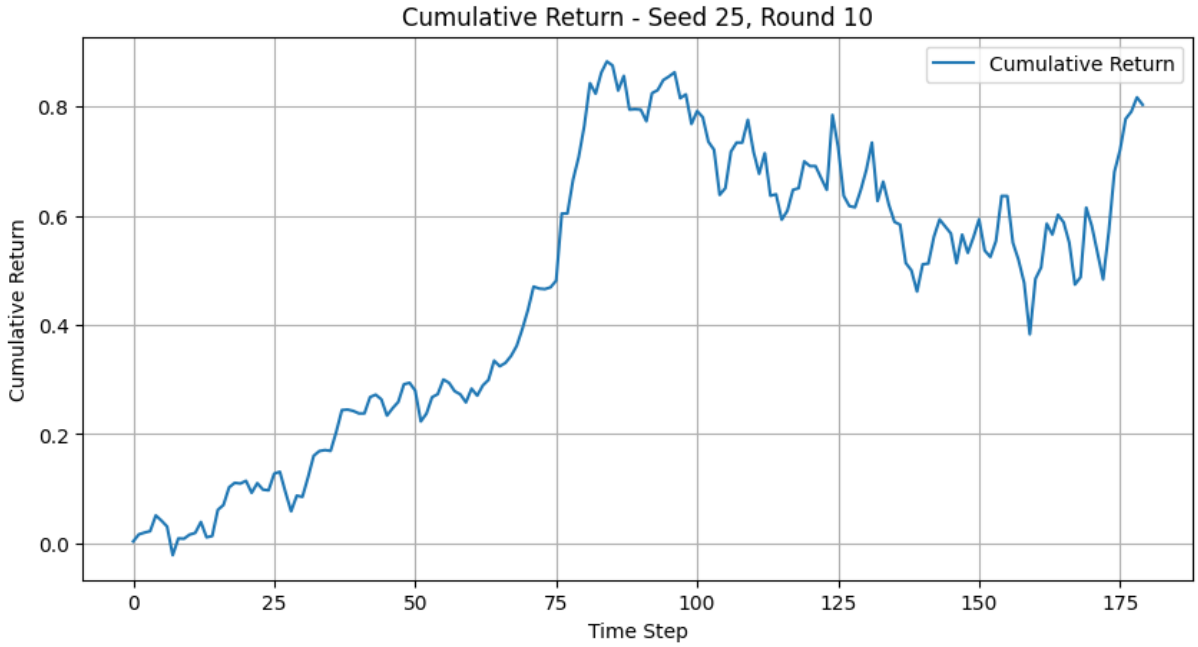


Figure 2 - Cumulative Portfolio Return Round 10 (Converged) -Training Set

Refer to Appendix D. for cumulative return plots of all 10 rounds.

The Sharpe ratios (Table 3) across 10 validation rounds range from 0.75 to 1.75, with a clear upward trend in later rounds. Early rounds (1–3) show relatively low Sharpe ratios, reflecting unstable or exploratory policies with higher volatility. Later rounds (7–10) yield consistently higher Sharpe ratios, indicating more stable and risk-aware portfolio strategies.

Round	Sharpe Ratio
1	0.8645
2	0.8162
3	0.7462
4	1.2315
5	1.1302
6	1.2267
7	1.4035
8	1.6340
9	1.3126
10	1.7522

Table 3 - Sharpe Ratio Across 10 Rounds

This improvement illustrates how the Recurrent PPO algorithm progressively learns to maximize risk-adjusted returns. The use of Sharpe ratio in the reward function encourages consistency, while penalties on allocation changes reduce over-trading, resulting in smoother and more effective policies over time.

Evolving Portfolio Allocations

In earlier rounds, portfolio allocations were volatile (Figure 3), showing random exploration across all five assets. Over time, the agent began consistently favoring top-performing assets, especially Asset 4 and Asset 5 (Figure 4). These allocation patterns suggest the emergence of a coherent trading strategy.

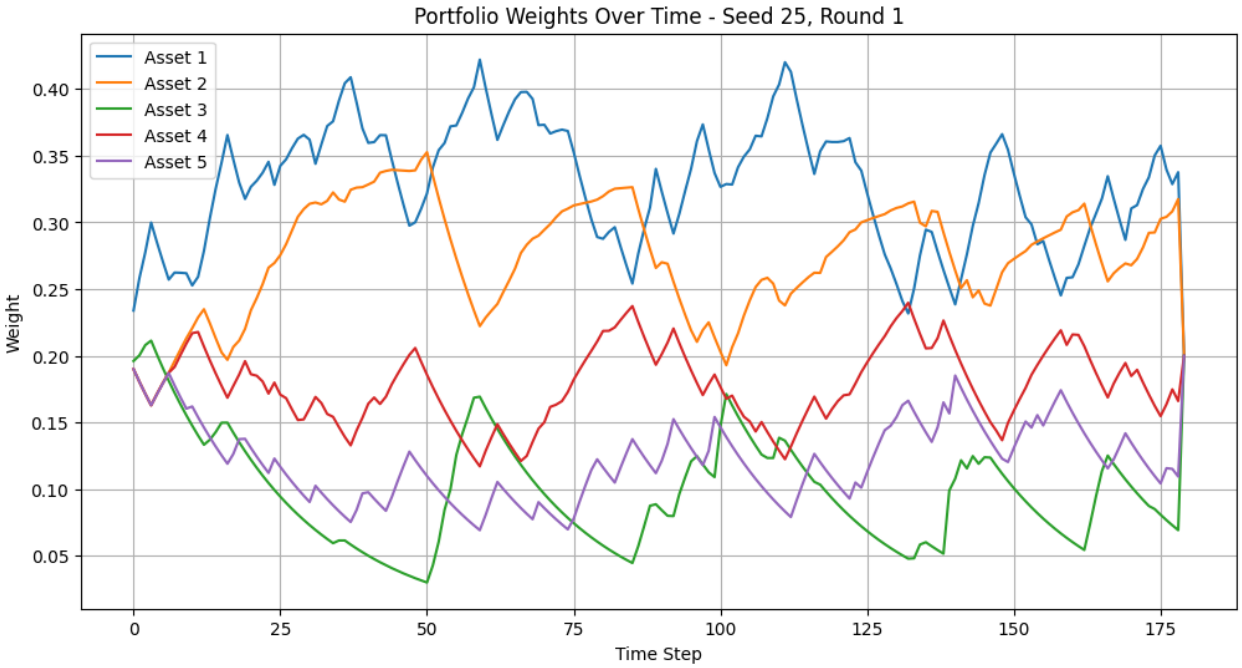


Figure 3 - Portfolio Weights Round 1 (Exploratory) - Training Set

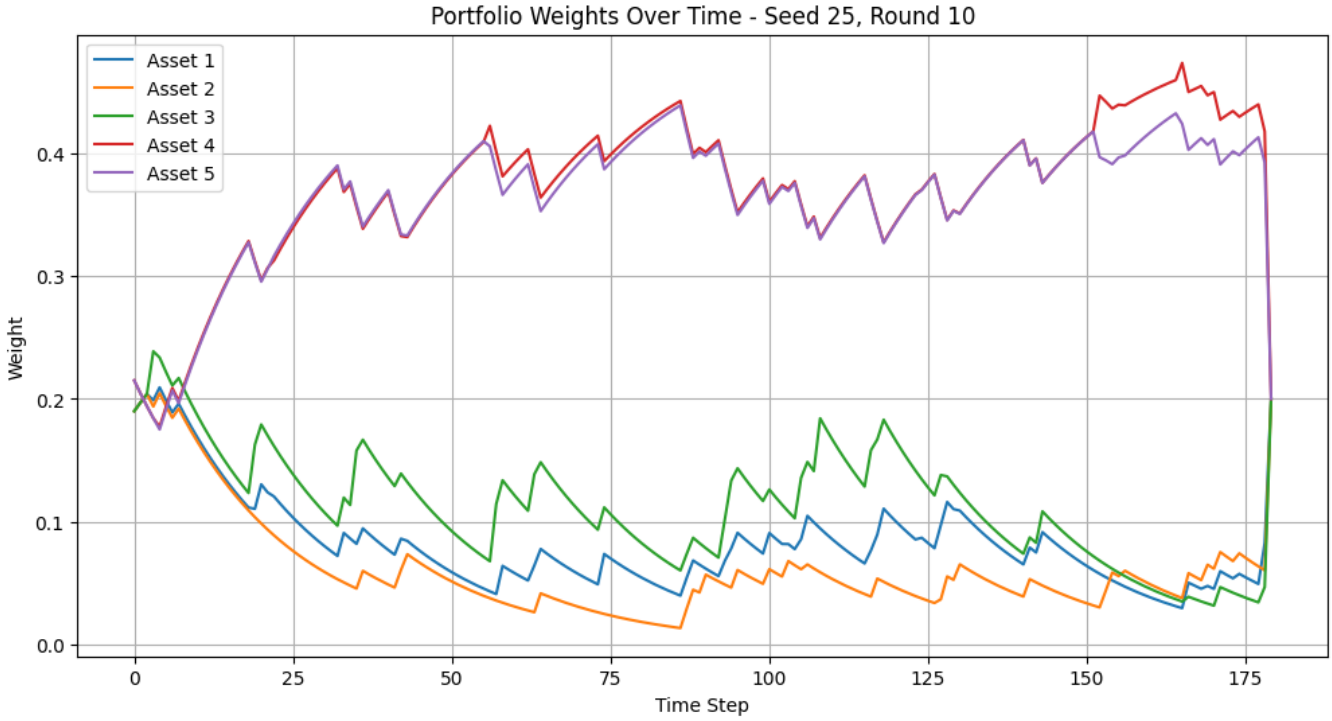


Figure 4 - Portfolio Weights Round 10 (Focused) - Training Set

Refer to Appendix E. for Portfolio Allocation Weight plots of all 10 rounds.

6.2 Learned Policy Behavior

The trained R-PPO model showed smooth transitions in portfolio weights across time, demonstrating a relatively stable allocation pattern that adapts to market shifts. The policy avoids excessive rebalancing, indicating the effectiveness of transaction cost penalties.

6.3 Test Set Evaluation

The learned policy was evaluated on the test set (01-01-2024 to 31-12-2024). The model effectively increased exposure to outperforming assets (Assets 4 and 5) while reducing allocation to declining ones (Figure 6). This adaptive allocation is evident from the smooth trajectory of portfolio weights over the test horizon.

- Sharpe Ratio: 1.4228

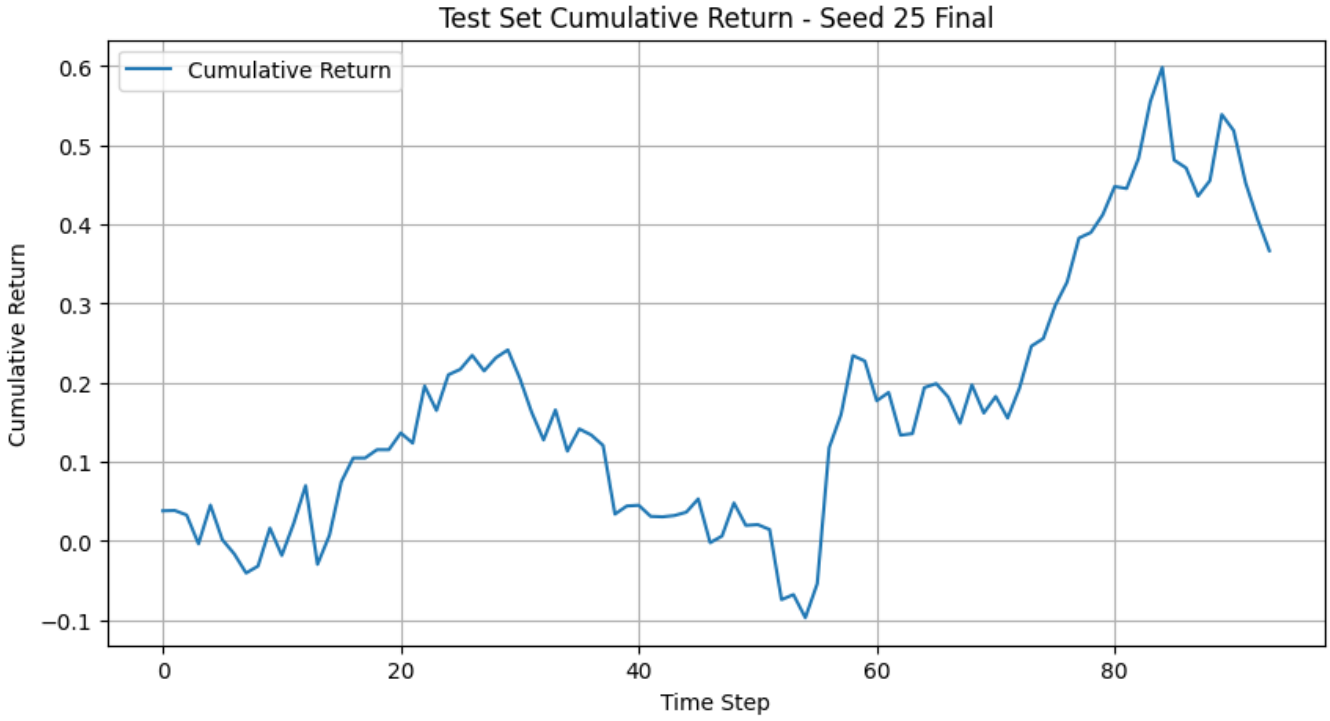


Figure 5 - Cumulative Returns - Test Set

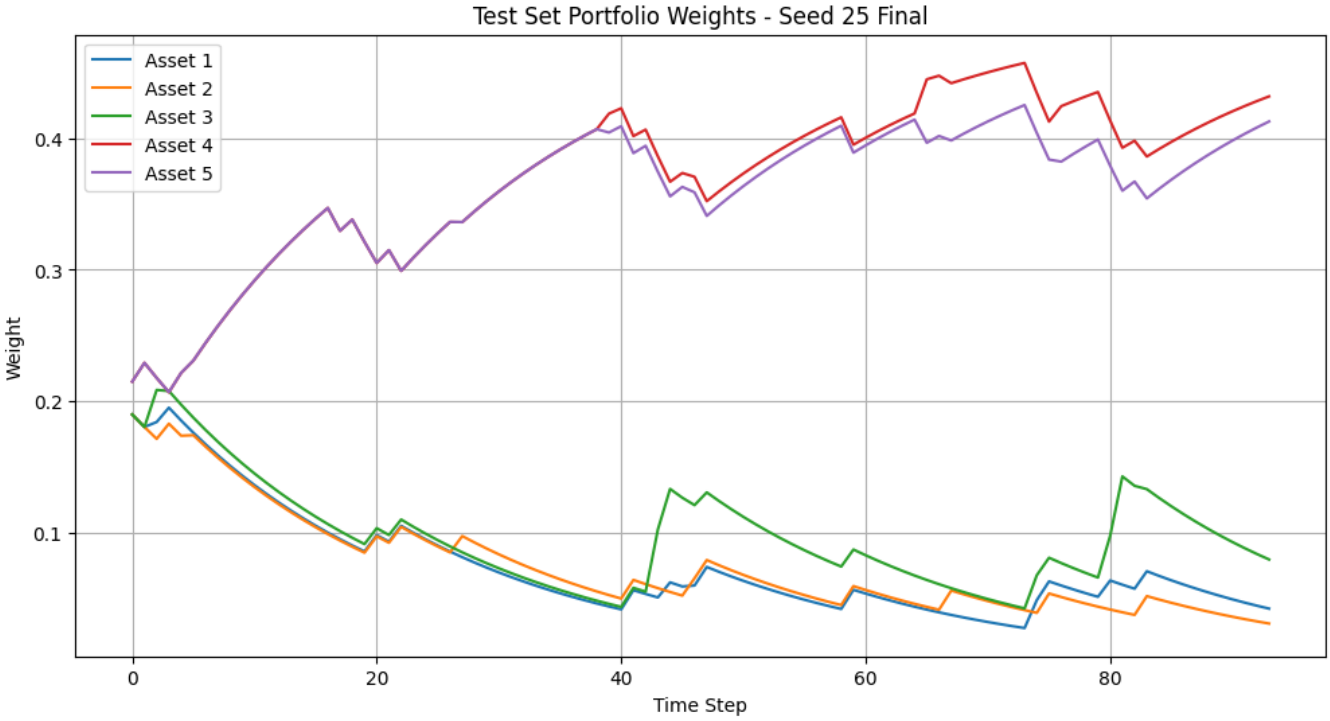


Figure 6 - Portfolio Weights - Test Set

6.4 Comparison with Baseline (Markowitz)

For benchmarking, we implemented a Markowitz Mean-Variance Optimization (MVO) model over the same dataset. The MVO portfolio was updated using a rolling window of 50 days, with zero transaction cost and no allocation constraints beyond normalization. Performance of the MVO policy on the 2024 test set:

- Sharpe Ratio: 0.3283

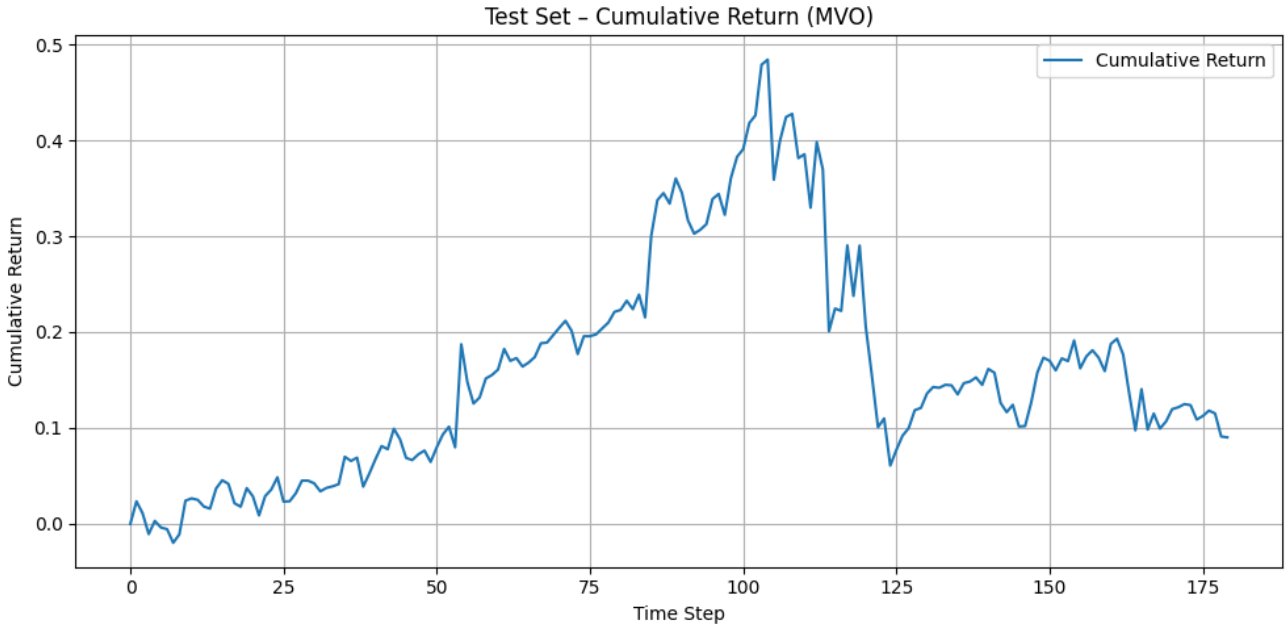


Figure 7 - Cumulative Returns - MVO - Test Set

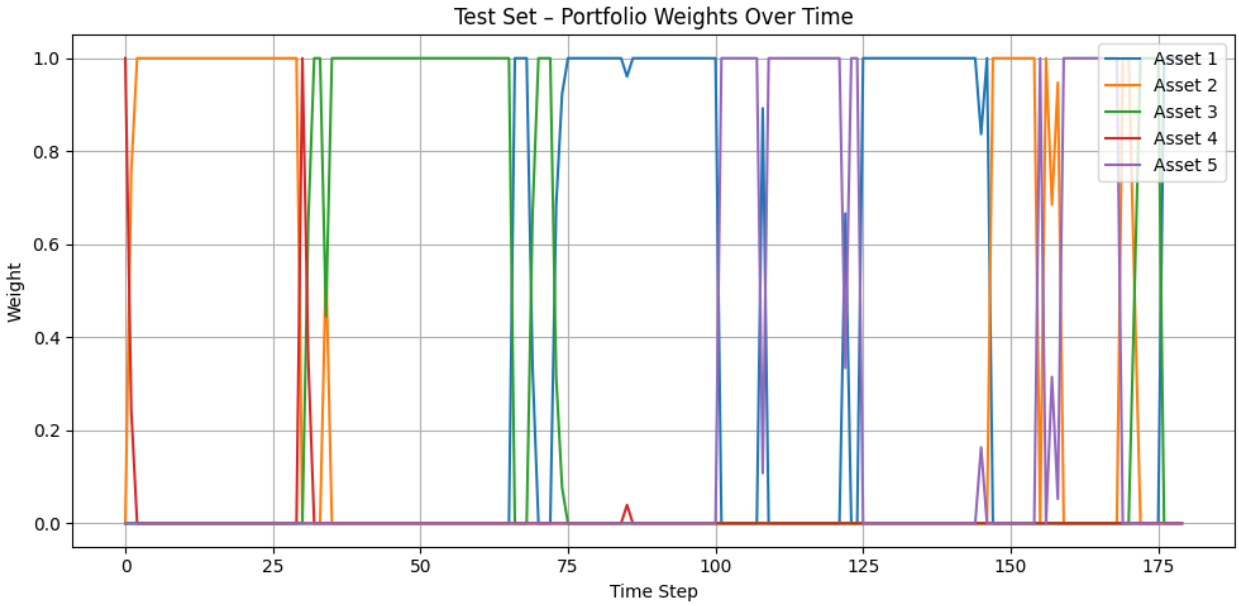


Figure 8 - Portfolio Weights - MVO - Test Set

The MVO model exhibited high turnover and non-smooth allocation shifts, contrasting with the stability of the RL-based policy. This highlights R-PPO's ability to learn temporal patterns and account for trading frictions.

7. Conclusion

This project explored the application of Reinforcement Learning (RL) for dynamic portfolio optimization, focusing specifically on the use of Recurrent Proximal Policy Optimization (R-PPO) to manage asset allocation across a portfolio of five major technology stocks. The environment was designed to realistically simulate financial trading by incorporating transaction costs, allocation change penalties, and reward signals driven by a combination of immediate returns and historical Sharpe ratios. The R-PPO algorithm demonstrated the ability to learn stable, risk-adjusted strategies that outperform traditional optimization methods.

On the 2024 test set, the learned R-PPO policy achieved a Sharpe Ratio of 1.42, significantly outperforming the baseline Markowitz Mean-Variance Optimization (MVO) model, which achieved a Sharpe Ratio of only 0.33. Unlike the MVO approach, which frequently switched between extreme allocations, the R-PPO agent maintained smoother and more diversified portfolio weights over time. This highlights the effectiveness of temporal pattern learning enabled by the LSTM policy network and the suitability of RL in handling non-stationary financial data.

Areas for Improvement:

Despite promising results, several aspects of the project offer room for further enhancement:

- **Model Generalization**: The model was trained and evaluated on a limited set of five assets. Future work could involve extending the portfolio size and assessing the scalability of R-PPO to more diverse asset classes.
- **Robustness Across Market Regimes**: While the model performed well on 2024 data, evaluating it across different market conditions (e.g., during crises or bull runs) could help test its robustness.
- **Transaction Cost Modeling**: A fixed transaction cost penalty was applied uniformly across assets. Incorporating realistic, asset-specific costs and slippage could further improve model fidelity.
- **Alternative RL Algorithms**: Although R-PPO was appropriate for the problem setting, algorithms like Soft Actor-Critic (SAC) or TD3 could be explored to compare performance in continuous action spaces under different exploration schemes.

- **Interpretability**: Reinforcement learning models often act as black boxes. Incorporating tools for interpretability, such as feature importance or action attribution, could enhance trust in the model's decisions.
- **Ensemble or Meta-Learning Approaches**: To stabilize performance across different seeds or datasets, an ensemble of policies or meta-RL techniques could be employed.

Overall, this work demonstrates that RL-based portfolio optimization is a viable and powerful alternative to classical techniques, particularly when market dynamics are complex, non-linear, and temporally dependent.

References

[1] H. M. Markowitz, “Portfolio Selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, Mar. 1952.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] W. F. Sharpe, “The Sharpe Ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994. doi: 10.3905/jpm.1994.409501

[5] J. Jiang, D. Xu, and J. Liang, “A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem,” *arXiv preprint arXiv:1706.10059*, 2017.

[6] Y. Moody and M. Saffell, “Reinforcement learning for trading,” in *Advances in Neural Information Processing Systems (NIPS)*, 2001, pp. 917–923.

[7] J. Schulman et al., “Proximal Policy Optimization Algorithms,” arXiv preprint arXiv:1707.06347, 2017.

[8] H. Heess et al., “Memory-based control with recurrent neural networks,” arXiv preprint arXiv:1512.04455, 2015.

[9] T. Lillicrap et al., “Continuous control with deep reinforcement learning,” arXiv:1509.02971, 2015.

[10] T. Haarnoja et al., “Soft Actor-Critic Algorithms for Deep Reinforcement Learning,” ICML 2018.

[11] S. Fujimoto et al., “Addressing Function Approximation Error in Actor-Critic Methods,” ICML 2018.

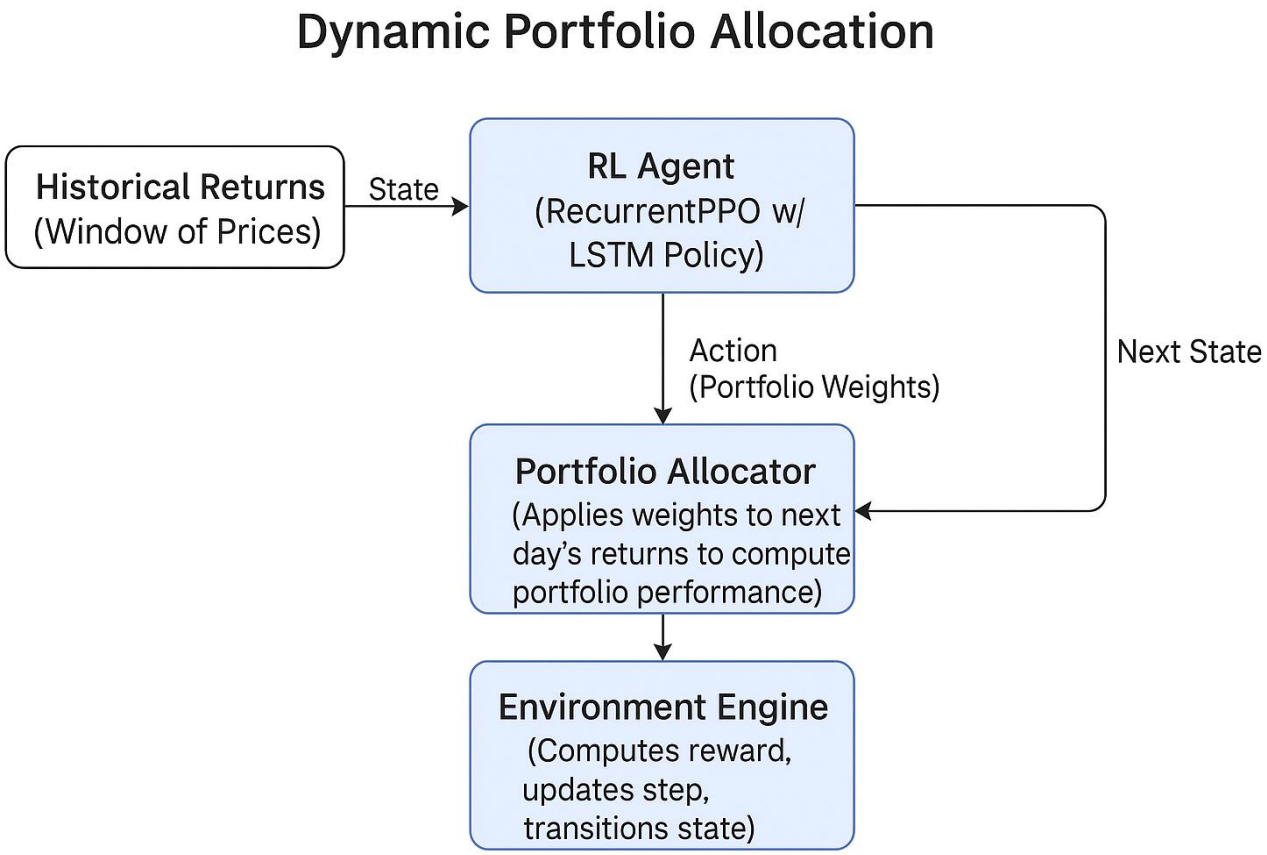
[12] R. Raffin, A. Hill, A. Gleave, M. Kanervisto, A. Ernestus, and N. Dormann, “Stable-Baselines3 Contrib,” *GitHub Repository*, 2021. [Online]. Available: https://github.com/Stable-Baselines-Team/stable-baselines3-contrib

[13] A. Raffin, A. Hill, A. Gleave, M. Kanervisto, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Open Source Software*, vol. 1, pp. 1–10, 2021.

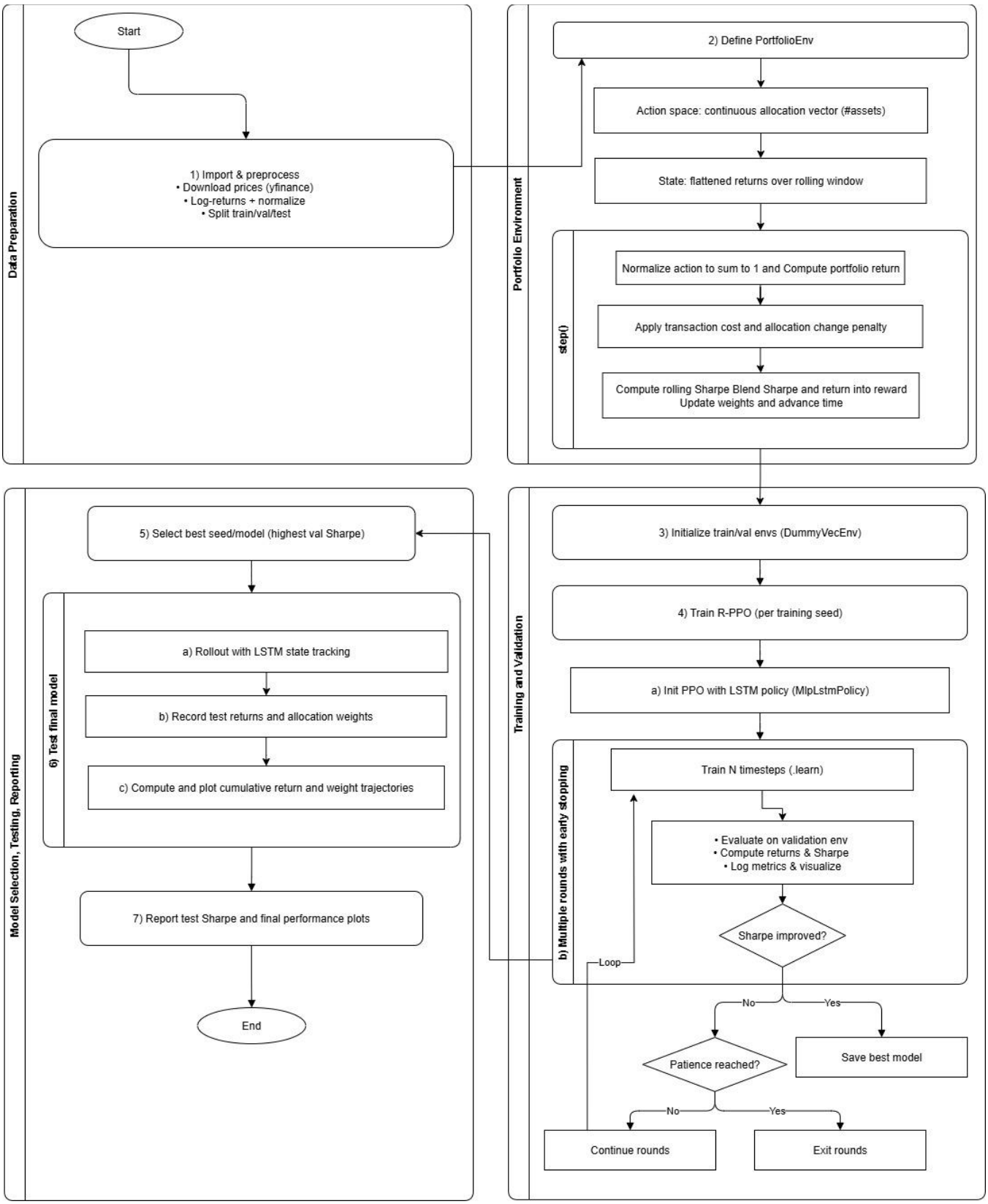
[14] G. Brockman et al., “OpenAI Gym,” *arXiv preprint*, arXiv:1606.01540, 2016.

Appendix

A. Recurrent PPO Architecture Diagram



B. Recurrent PPO Workflow Diagram

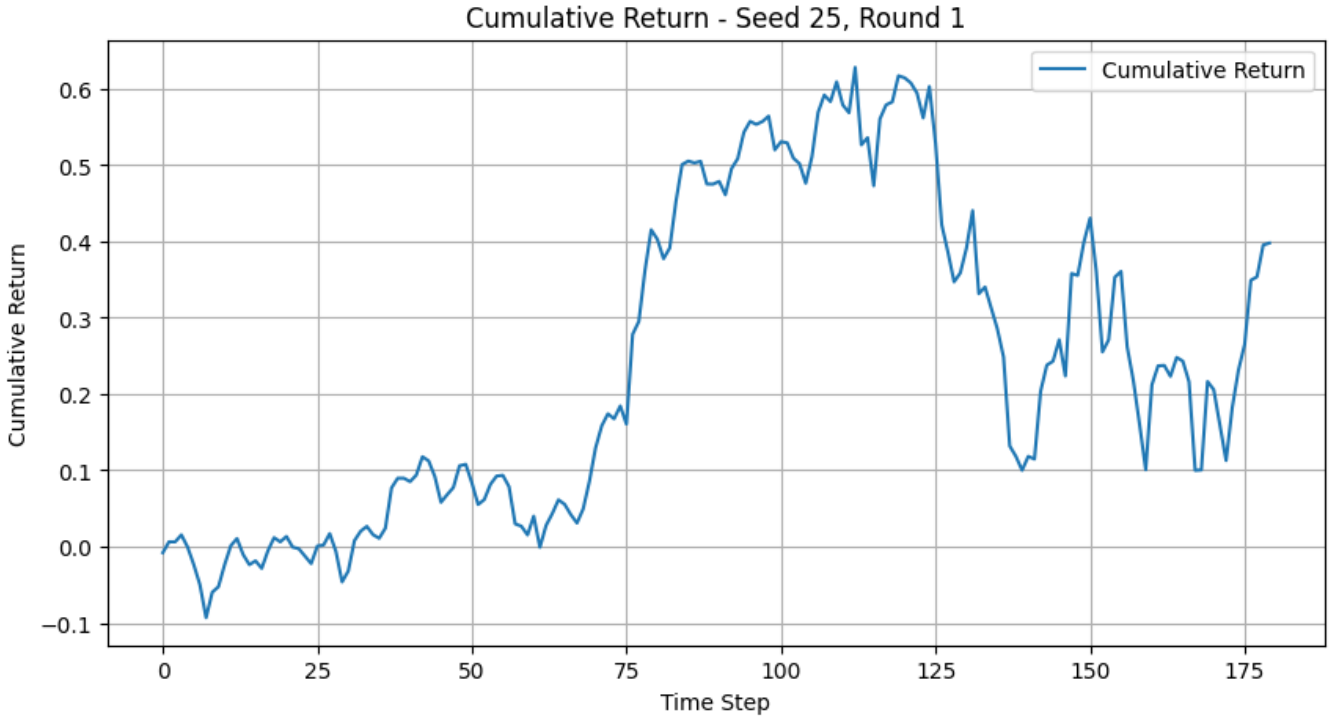


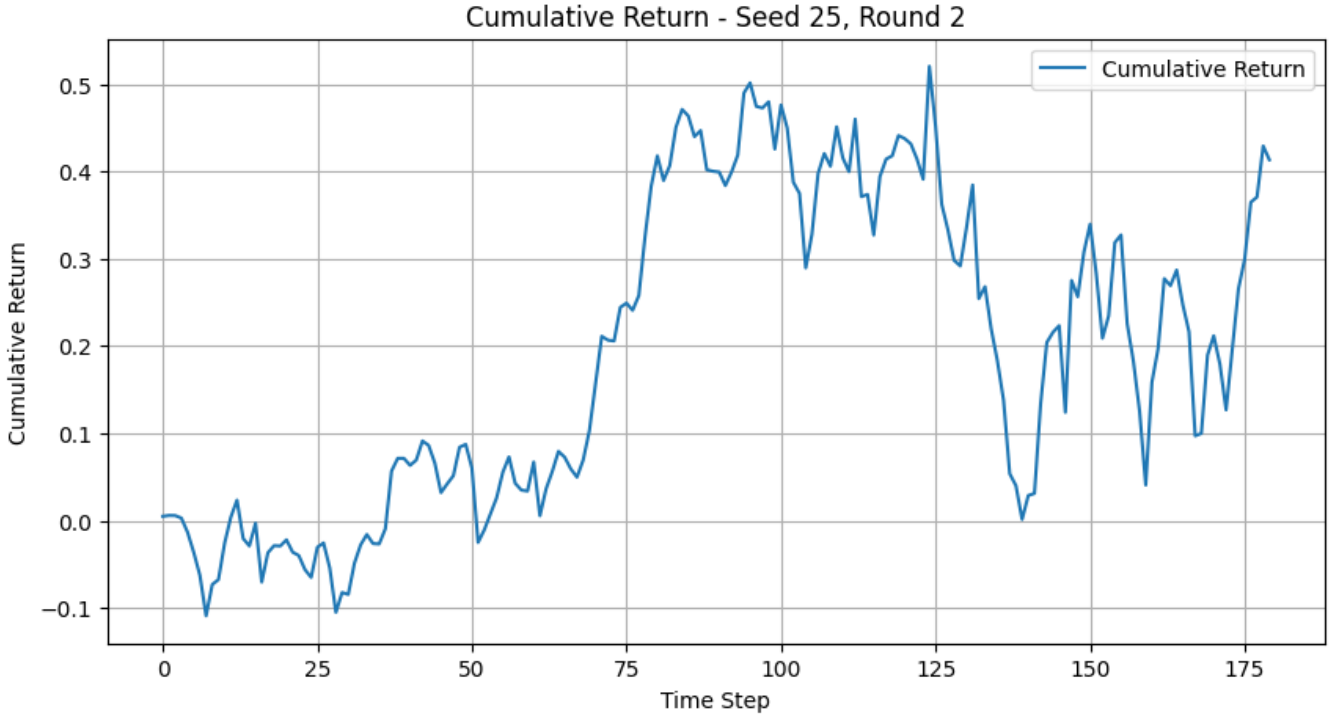
C. Environment Setup & Key Hyperparameters

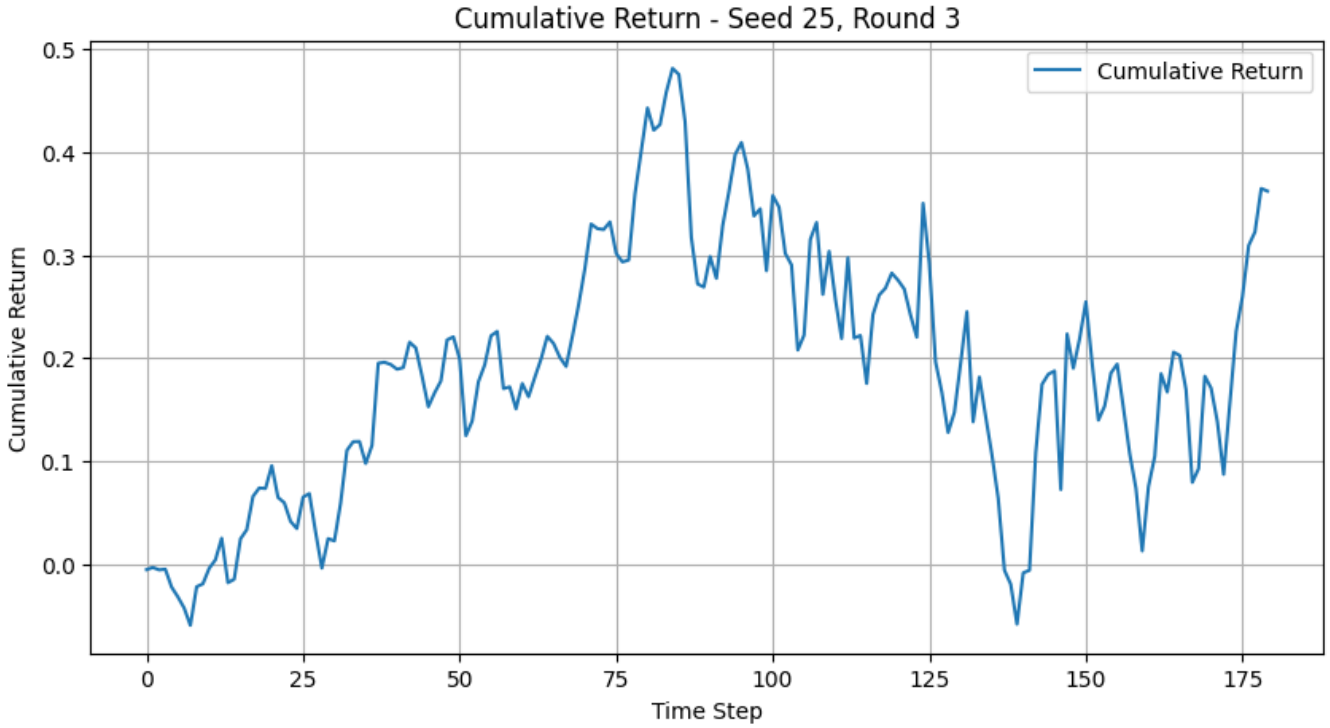
Category	Parameter	Value / Description
Environment	Number of Assets	5
	Observation Window Size	25 Trading Days (rolling window of log returns)
	Sharpe Window	50 Trading Days
	Transaction Cost	0.001 per unit change in allocation
	Allocation Change Penalty	0.1 * sum((action - weights)^2)
	Episode Length	180 Trading Days
	Weights Update Rule	w_t = 0.95 * w_t + 0.05 * action_t
Model Architecture	Policy Network	LSTM-based PPO
	Input Shape	Flattened state vector of shape (window size × number of assets)
	Output	Allocation vector of size 5
Training Settings	Algorithm	Proximal Policy Optimization (Recurrent PPO via Stable-Baselines3 + LSTM)
	Frameworks Used	Stable-Baselines3, PyTorch, Gym, NumPy, Matplotlib
	Timesteps per rollout (n_steps)	128
	PPO Epochs (n_epochs)	10
	Learning Rate	1e-4
	Discount Factor (γ)	0.99
	GAE Lambda	0.95
	Clip Range	0.2
	Batch Size	64
	Entropy Coefficient	0.01

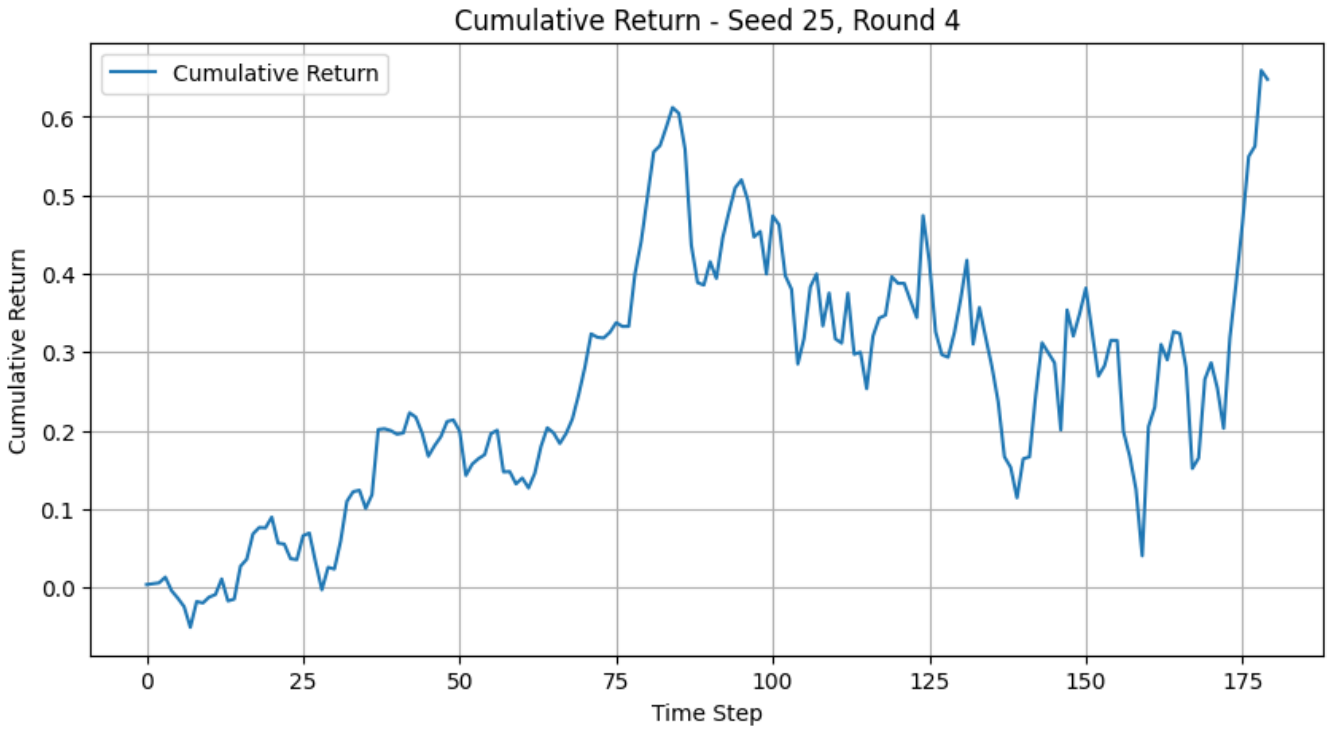
Category	Parameter	Value / Description
	Value Function Coefficient	0.5
	Max Grad Norm	0.5
	Early Stopping	Based on Sharpe Ratio on Validation set
Backtesting	Train Period	Jan 2015 – Dec 2020
	Validation Period	Jan 2021 – Dec 2023
	Test Period	Jan 2024 – Dec 2024
	Test Set Sharpe Ratio (PPO)	**1.4228**
	Test Set Sharpe Ratio (MVO Baseline)	**0.3283**

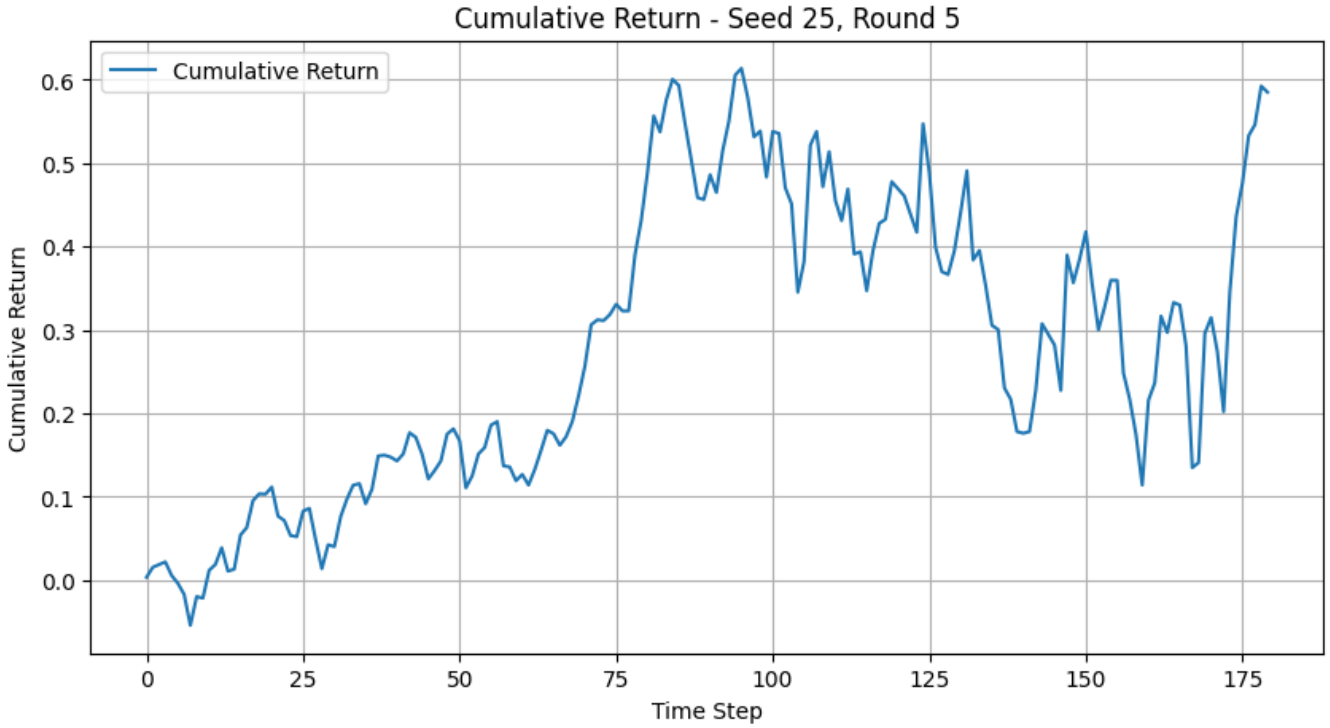
D. Cumulative Return Plots for 10 Training Rounds

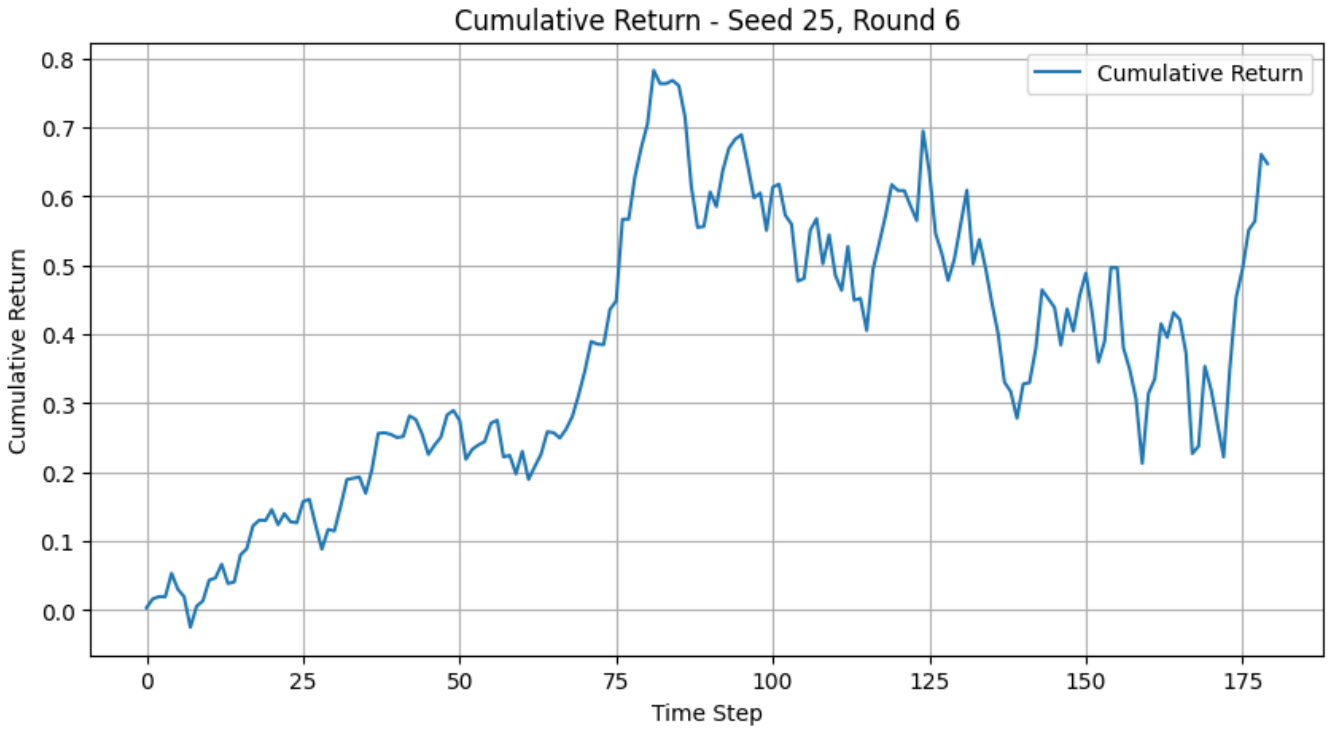


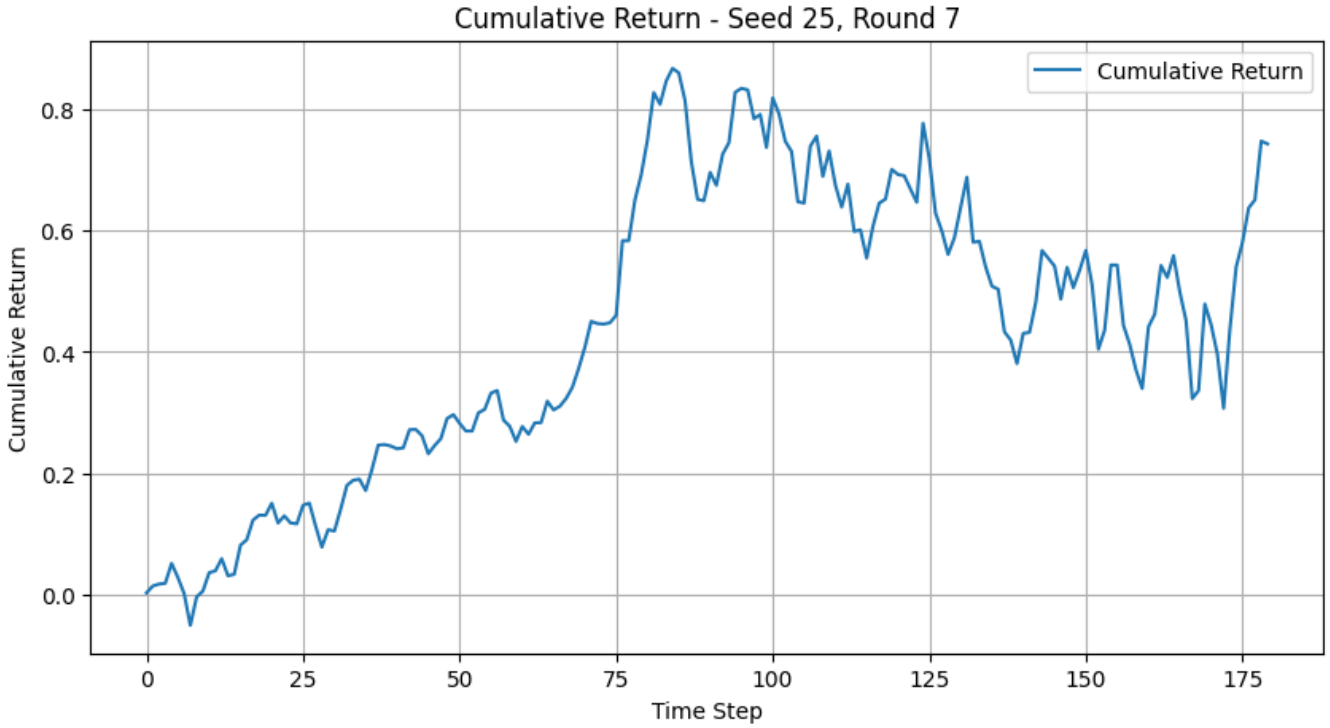


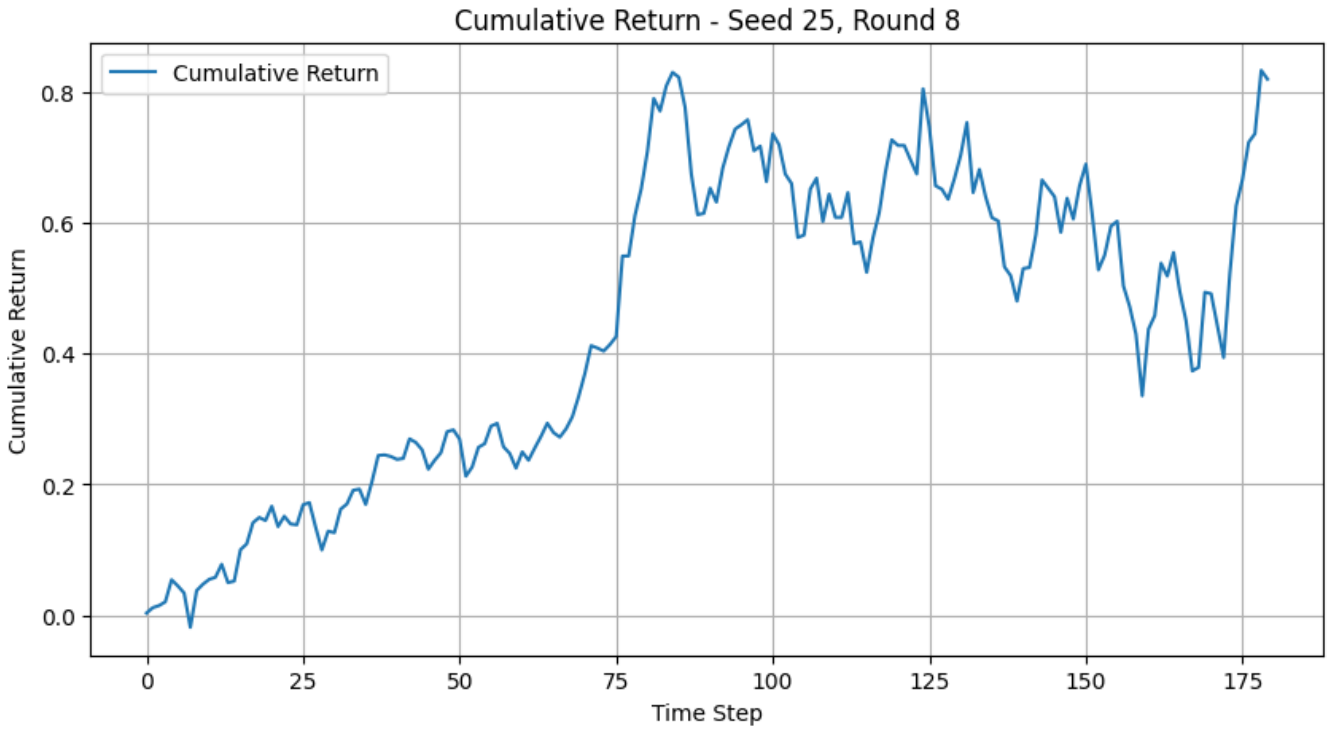

Cumulative Return - Seed 25, Round 3
Cumulative Return
Cumulative Return
Time Step
0.5
0.4
0.3
0.2
0.1
0.0
0
25
50
75
100
125
150
175

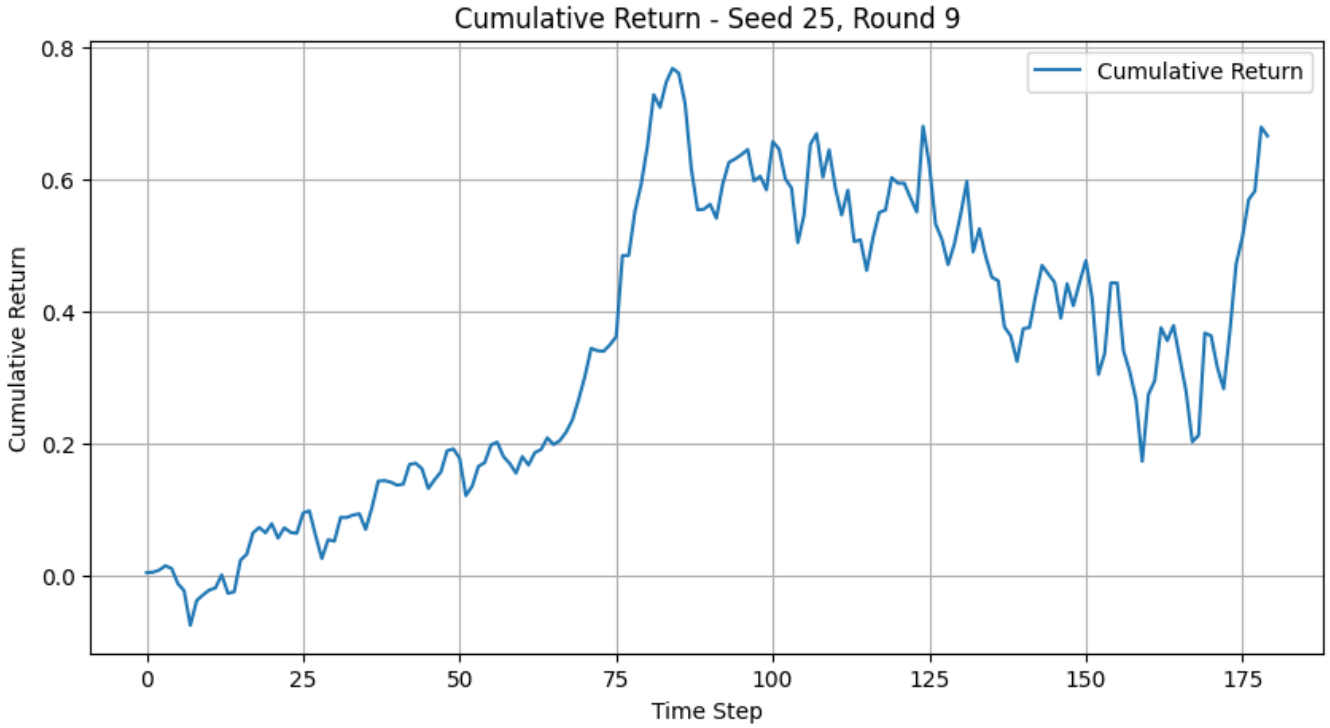

Cumulative Return - Seed 25, Round 4
Cumulative Return
Cumulative Return
Time Step
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0
25
50
75
100
125
150
175

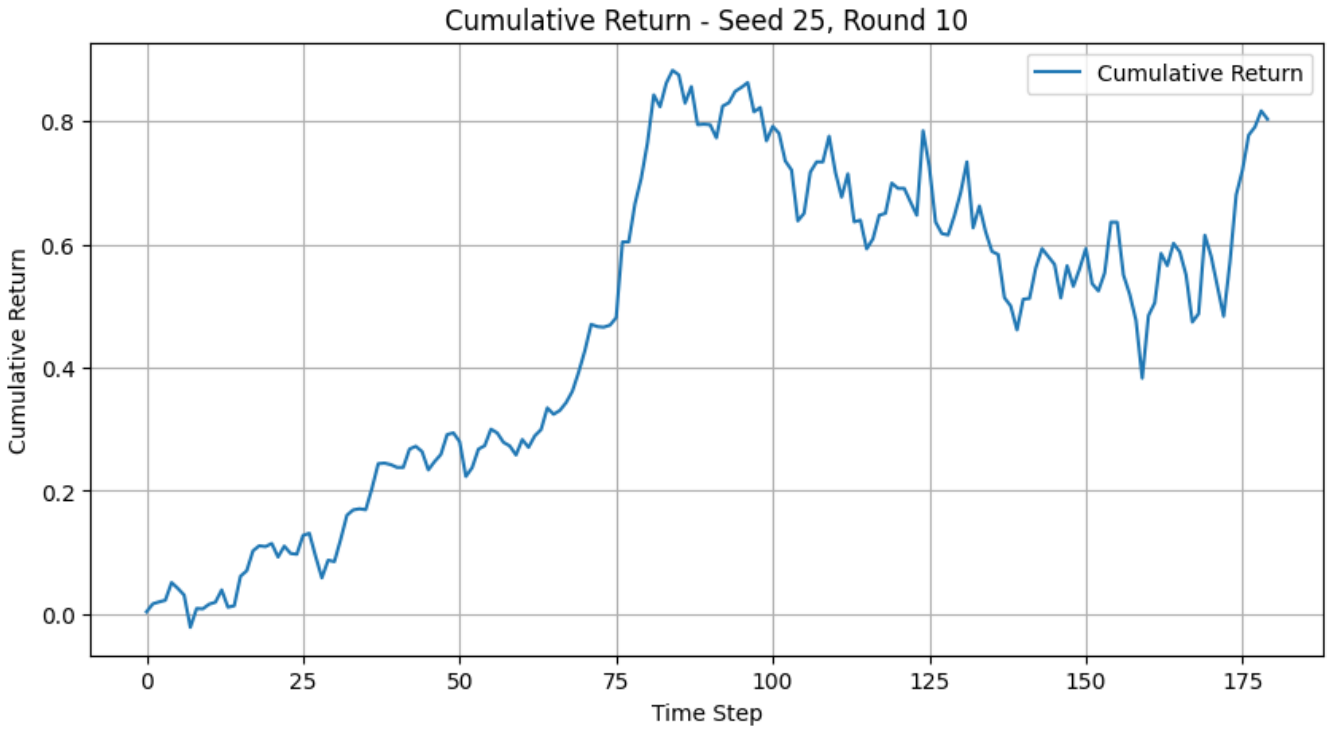

Cumulative Return - Seed 25, Round 5
Cumulative Return
Cumulative Return
Time Step
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0
25
50
75
100
125
150
175


Cumulative Return - Seed 25, Round 6
Cumulative Return
Cumulative Return
Time Step
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
25
50
75
100
125
150
175

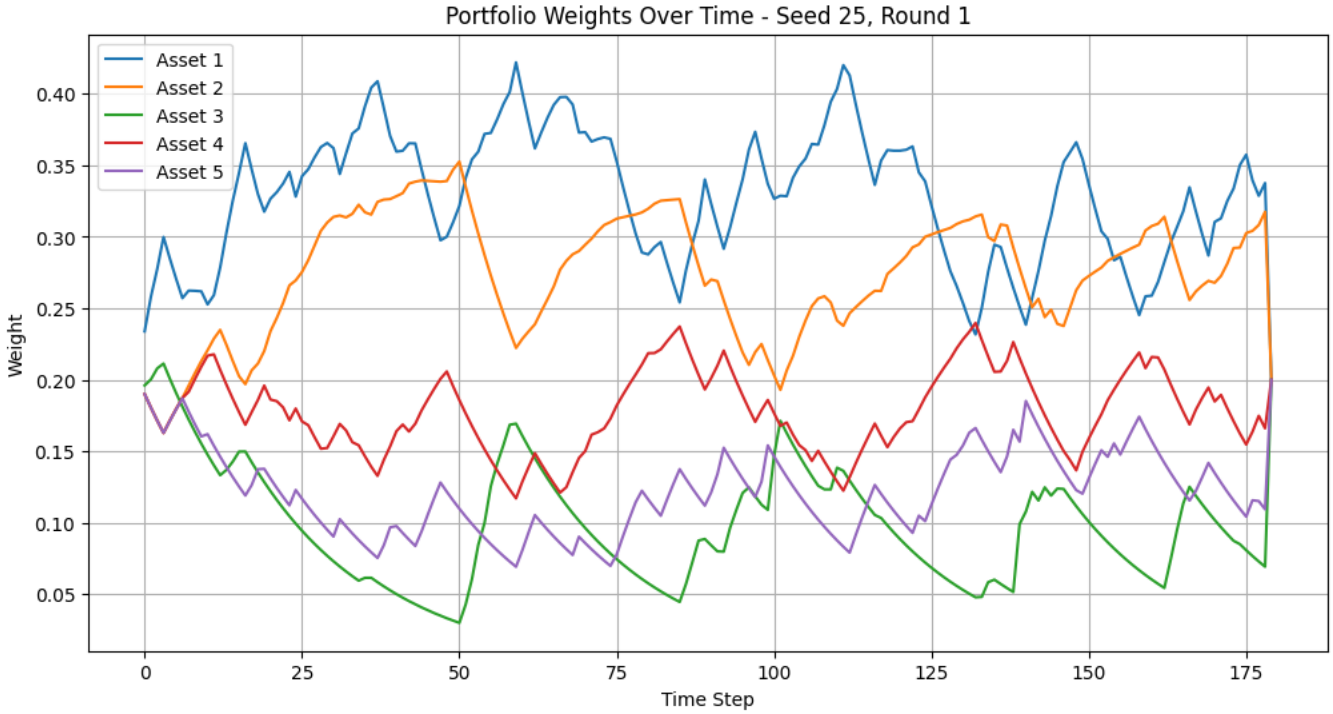

Cumulative Return - Seed 25, Round 7
Cumulative Return
Cumulative Return
0.0
0.2
0.4
0.6
0.8
0
25
50
75
100
125
150
175
Time Step

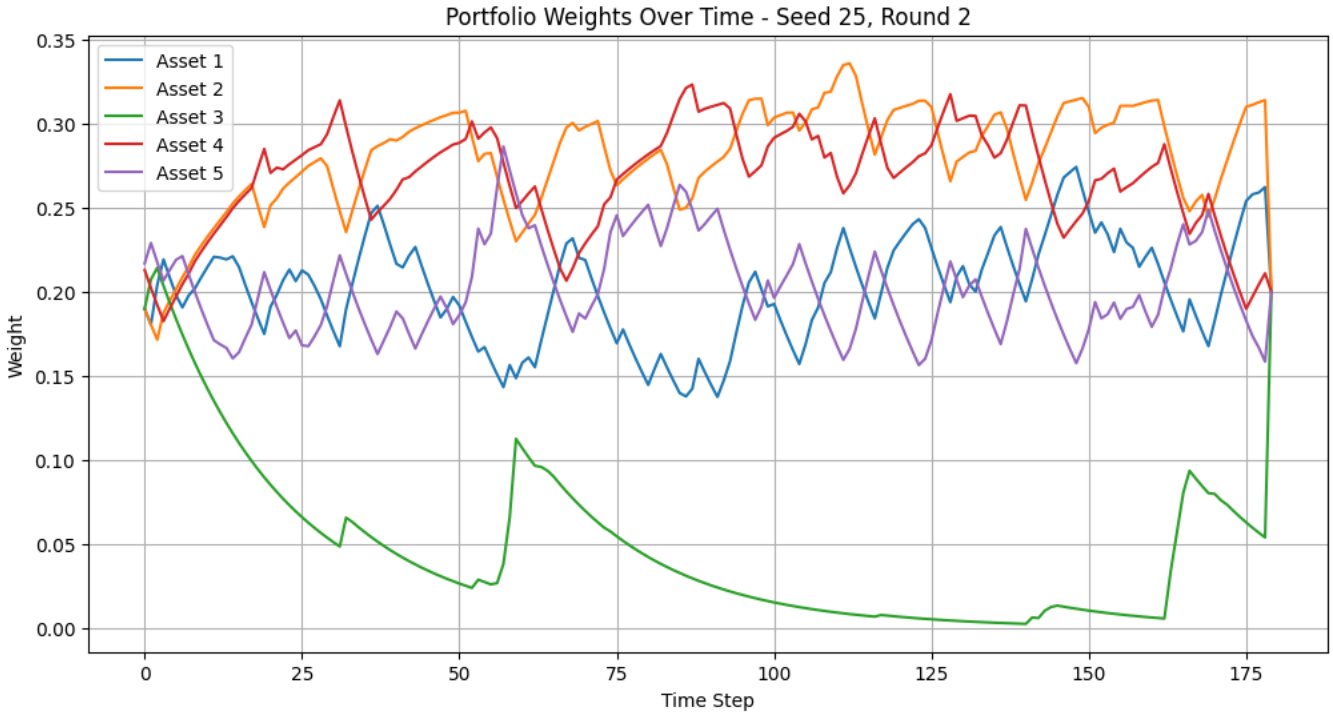

Cumulative Return - Seed 25, Round 8
Cumulative Return
Cumulative Return
0.0
0.2
0.4
0.6
0.8
0
25
50
75
100
125
150
175
Time Step

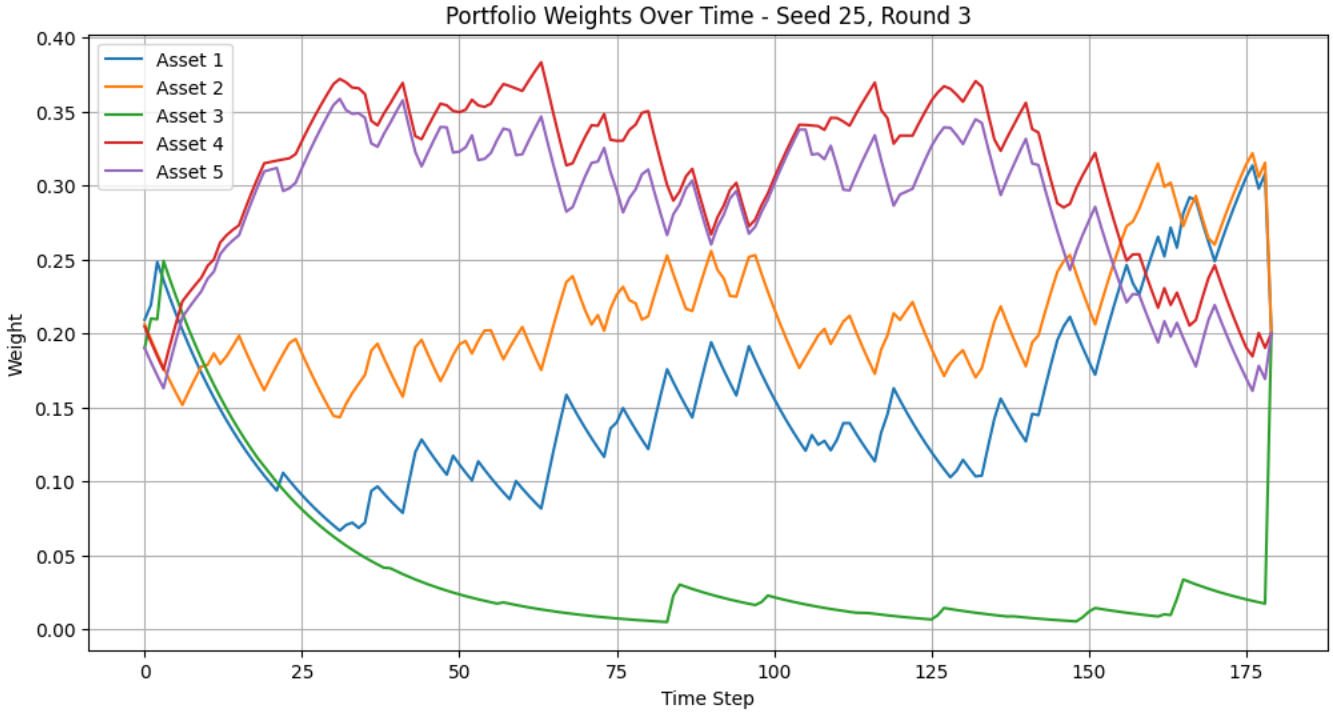

Cumulative Return - Seed 25, Round 9
Cumulative Return
Cumulative Return
0.8
0.6
0.4
0.2
0.0
0
25
50
75
100
125
150
175
Time Step

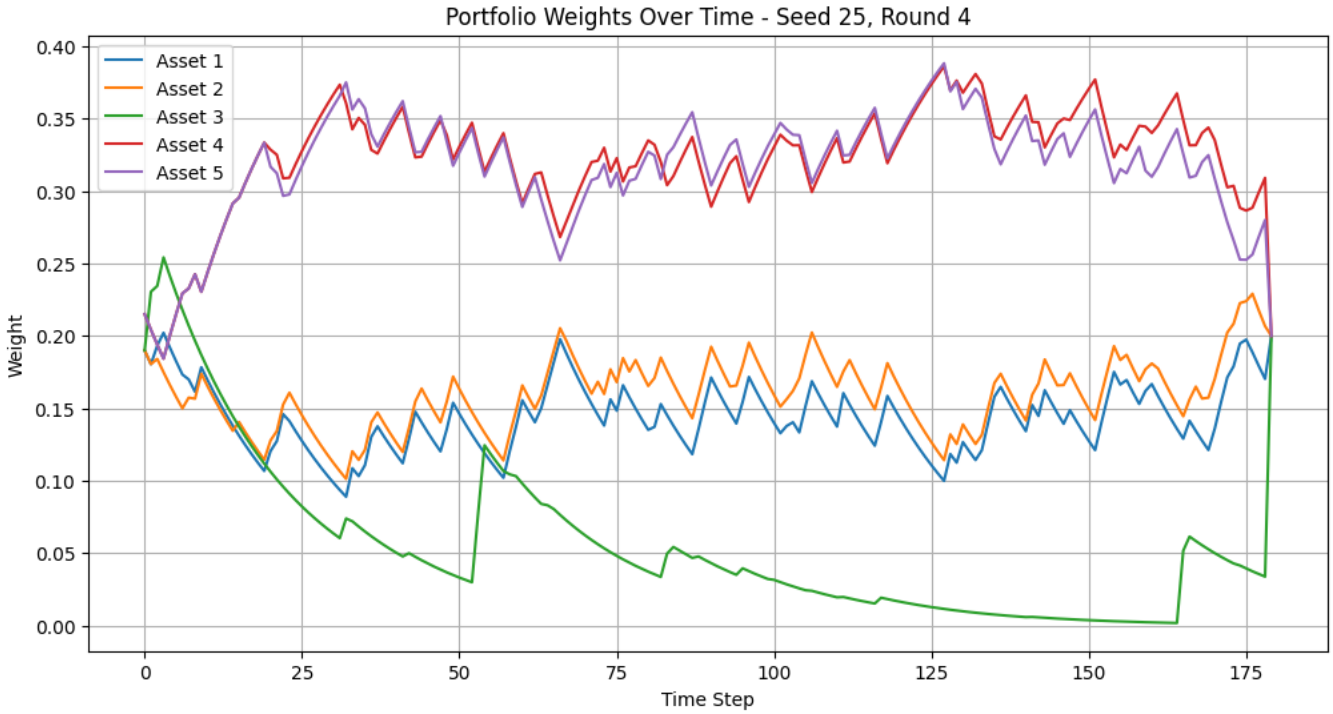

Cumulative Return - Seed 25, Round 10
Cumulative Return
Cumulative Return
0.8
0.6
0.4
0.2
0.0
0
25
50
75
100
125
150
175
Time Step

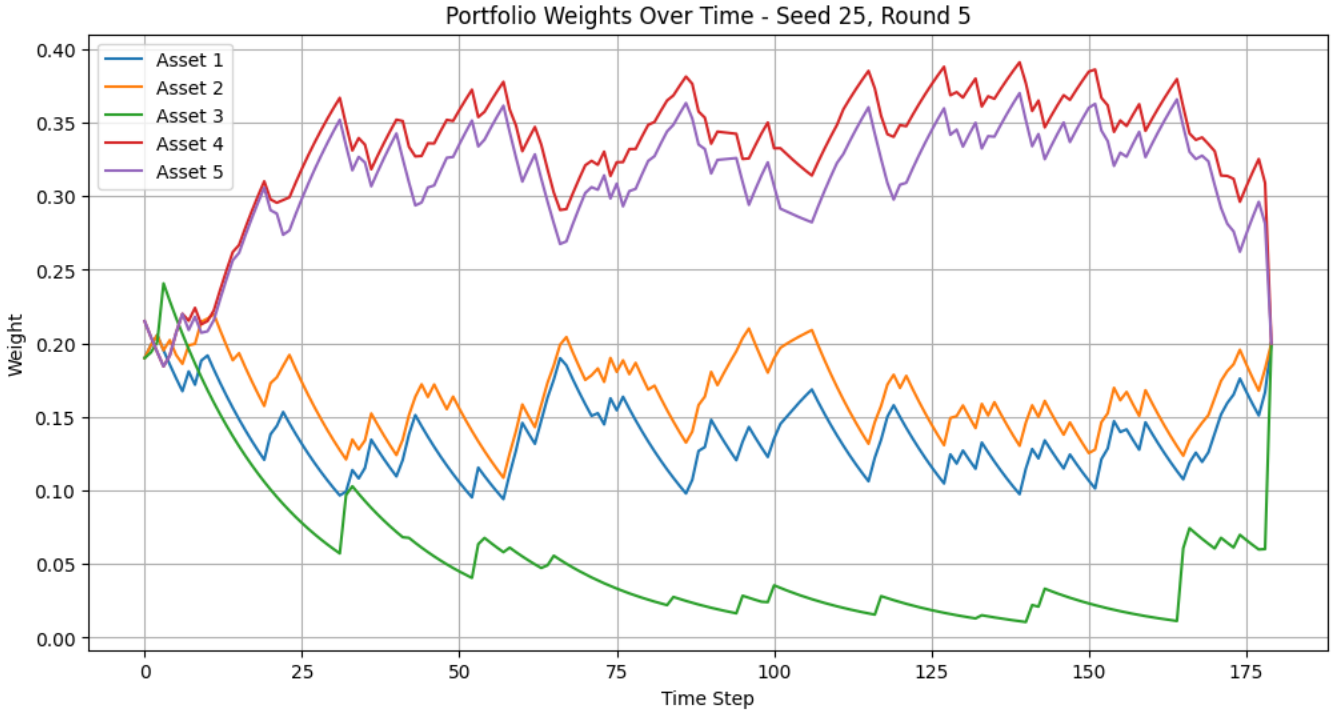
E. Portfolio Weights Plots for 10 Training Rounds

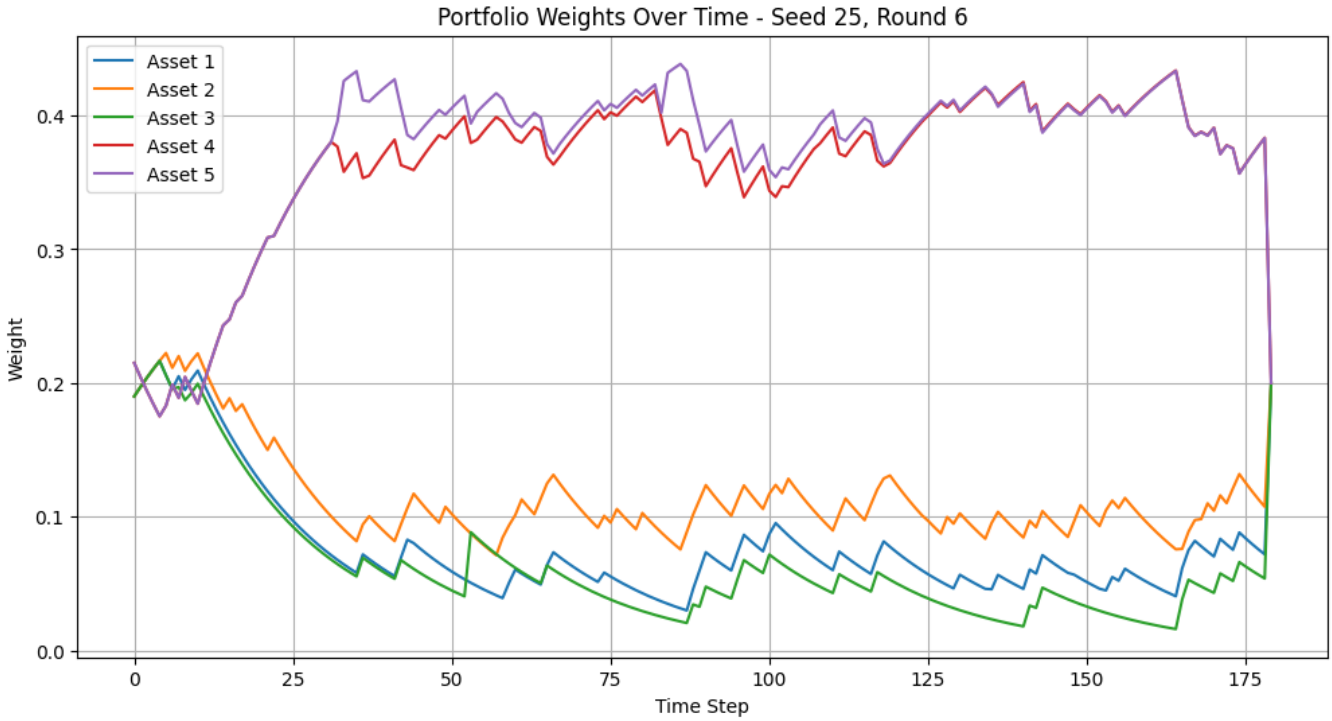


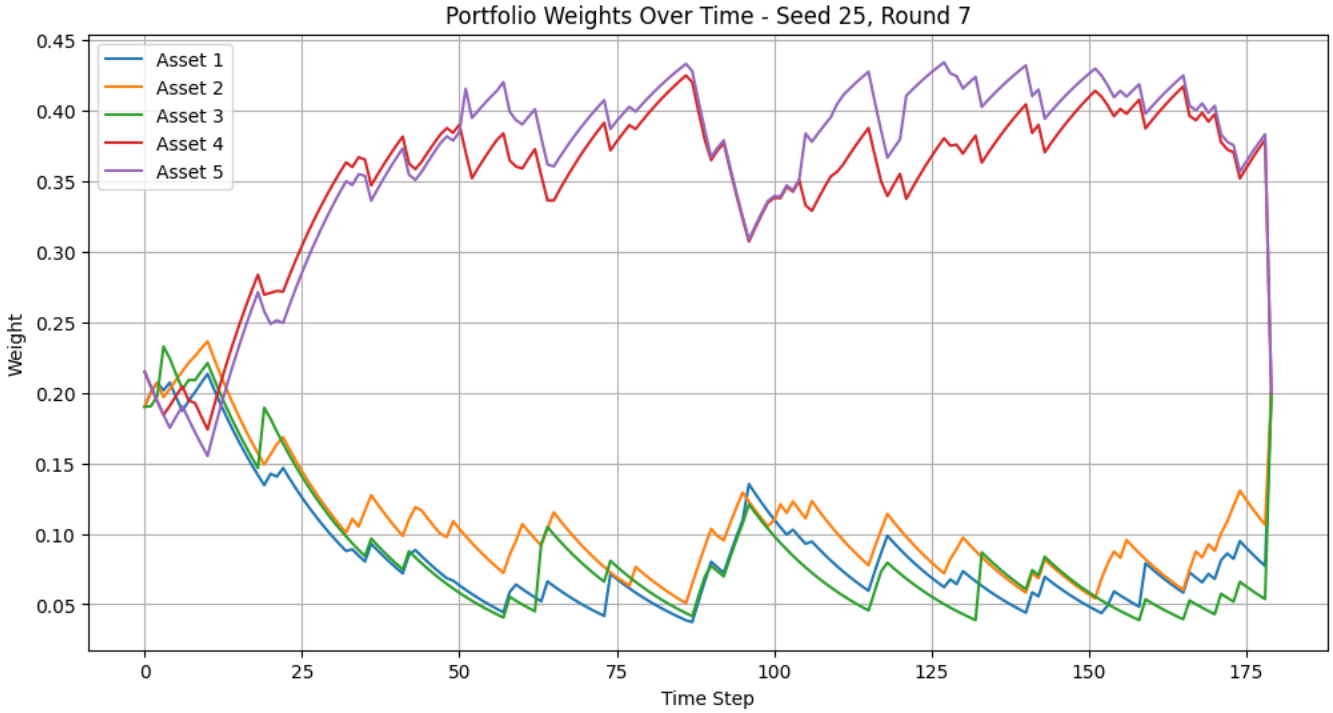


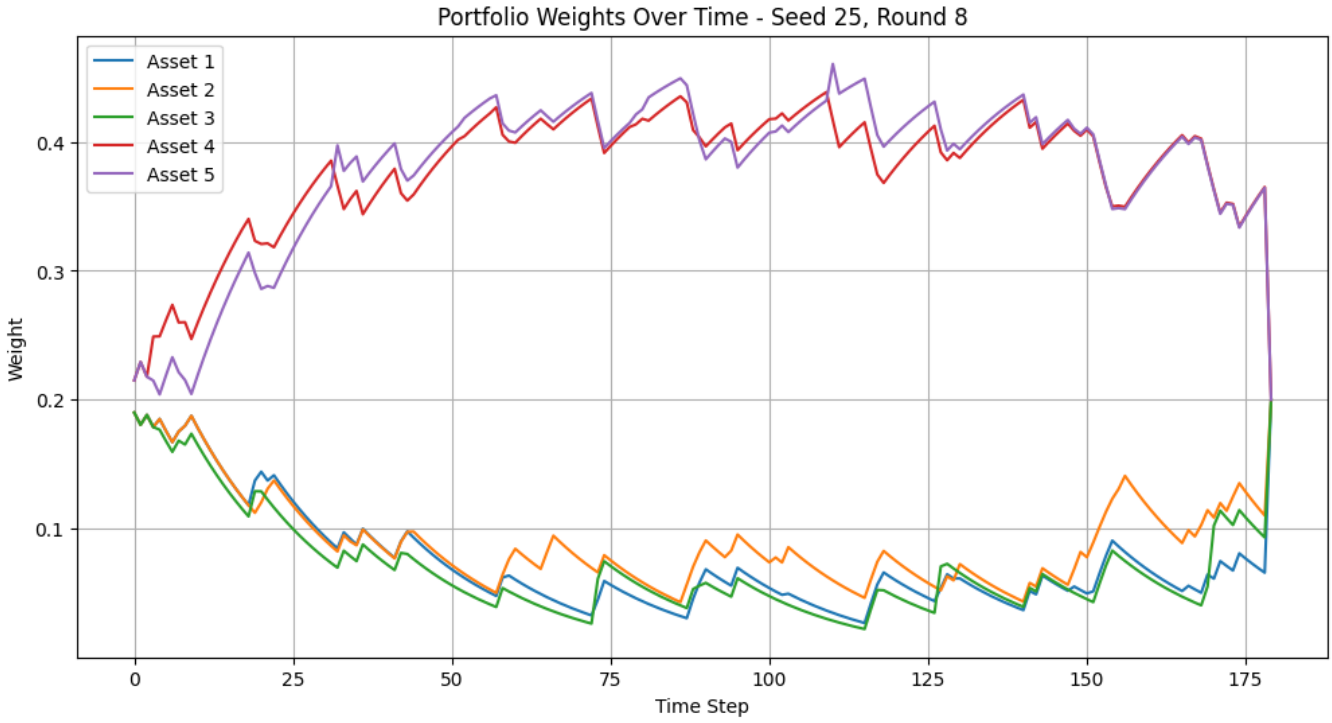

Portfolio Weights Over Time - Seed 25, Round 3
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
Time Step
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
0
25
50
75
100
125
150
175

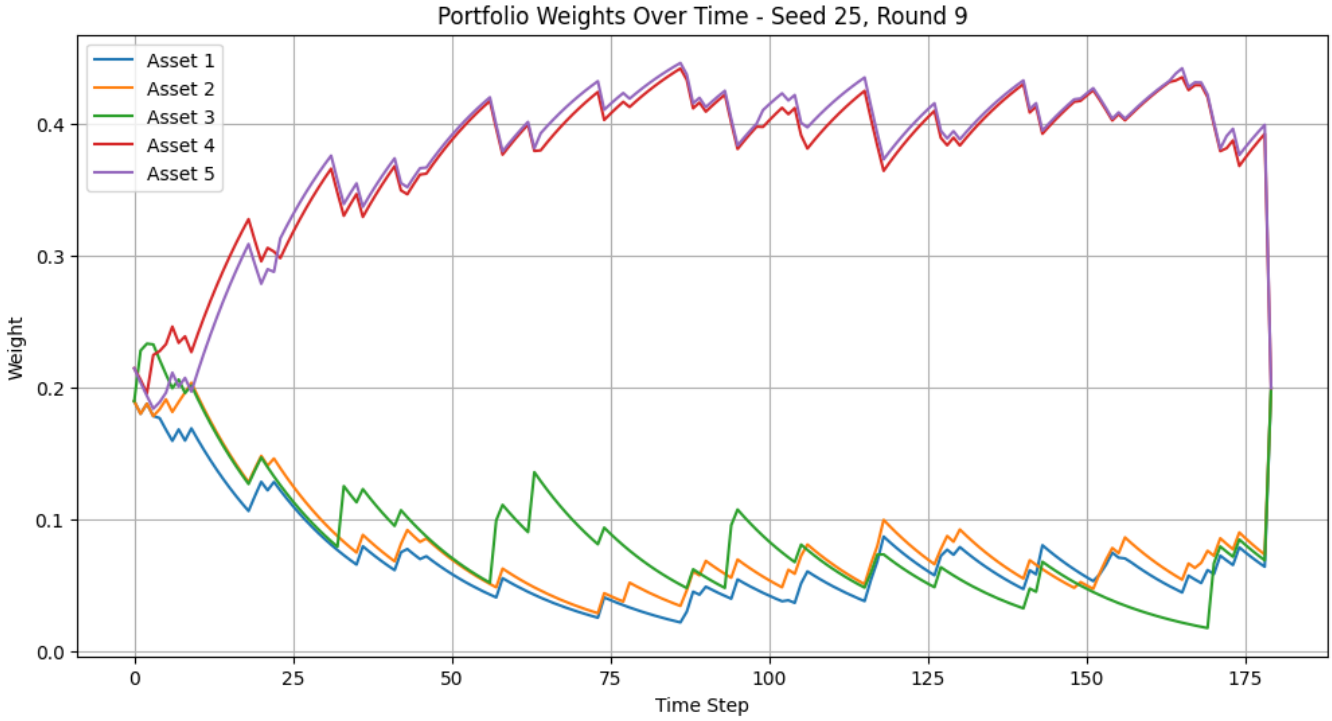

Portfolio Weights Over Time - Seed 25, Round 4
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
Time Step
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
0
25
50
75
100
125
150
175

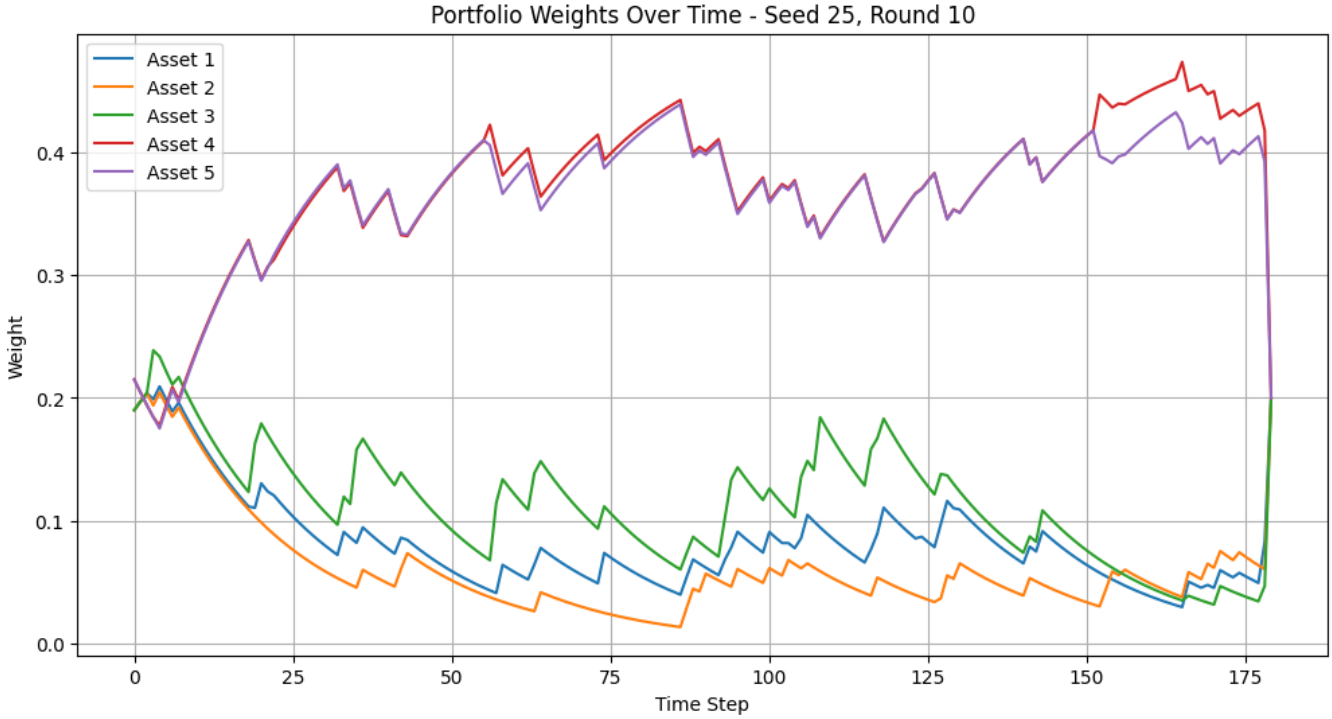

Portfolio Weights Over Time - Seed 25, Round 5
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
Time Step
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
0
25
50
75
100
125
150
175


Portfolio Weights Over Time - Seed 25, Round 6
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
Time Step
0.0
0.1
0.2
0.3
0.4
0
25
50
75
100
125
150
175


Portfolio Weights Over Time - Seed 25, Round 7
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
Time Step
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0
25
50
75
100
125
150
175


Portfolio Weights Over Time - Seed 25, Round 8
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
Time Step
0.4
0.3
0.2
0.1
0
25
50
75
100
125
150
175


Portfolio Weights Over Time - Seed 25, Round 9
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
0.0
0.1
0.2
0.3
0.4
0
25
50
75
100
125
150
175
Time Step


Portfolio Weights Over Time - Seed 25, Round 10
Asset 1
Asset 2
Asset 3
Asset 4
Asset 5
Weight
0.0
0.1
0.2
0.3
0.4
0
25
50
75
100
125
150
175
Time Step